Quantum-Inspired Optimization for Entanglement Routing in Quantum Networks: A Technical Report

Full Mathematical Derivations, Solver Architectures, and Experimental Analysis

Arnav Kapoor
QiOpt4QNet Project
https://github.com/arnavk23/QiOpt4QNet-Simulator

August 14, 2026

Contents

1 Introduction and Motivation

Quantum networks distribute entanglement—the Bell-state correlations $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$—between distant nodes [1, 2, 3]. Unlike classical networks, where a bit is a robust two-state system, entanglement is a fragile resource that decays exponentially with time (T1/T2 decoherence) and must be continuously regenerated via probabilistic processes (entanglement swapping with success probability $p_{\text{swap}} \ll 1$, BBPSSW purification rounds) [9, 8]. The entangled resource underpins quantum teleportation [11] and quantum key distribution [13, 14, 15, 16]; at the scale of a full network it requires repeaters [4, 5, 6, 7] and careful routing [18, 19, 20, 21, 22, 23].

The core optimization problem is: given a set of concurrent entanglement requests $\mathcal{R} = \{r_1, \ldots, r_M\}$ between source–destination pairs (s_r, d_r) on a network graph $G = (V, E)$, select for each request a path p_r and purification schedule q_r (a *bundle*) that maximizes end-to-end fidelity while respecting edge and memory capacities. This is a constrained combinatorial optimization problem in $\mathcal{O}(|\mathcal{R}|^2)$ decision variables, which we formulate as a Quadratic Unconstrained Binary Optimization (QUBO) problem [31, 32] and solve with six quantum-inspired solvers.

This report provides a complete mathematical treatment of:

- The quantum network model (Sec. 2)
- The QUBO Hamiltonian with all penalty and physical terms (Sec. 3)
- Six solver architectures—Metropolis annealer, tensor-network (MPS) compressor, streaming annealer, OpenJij simulated annealer, an embedded quantum-annealing backend, and a hybrid pipeline with GNN-guided candidate reduction (Sec. 4)
- A stochastic reliability framework: discrete-event simulation (DES), chance-constrained routing, recourse, and optimality certification against exact integer programming (Sec. 5)
- Analysis extensions: purification as a first-class variable, multi-objective and disjoint-path provisioning, swapping-order scheduling, and generative topologies (Sec. 6)
- A comprehensive experimental benchmark with interpretation (Sec. 8)
- A cross-domain comparison of QKD routing and entanglement routing assumptions (Sec. 9)

The codebase is fully open source at `https://github.com/arnavk23/QiOpt4QNet-Simulator` and all results in this report are reproducible via `src/experiments/experiment\_suite.py`.

2 Quantum Network Model

2.1 Network Graph

The quantum network is an undirected graph $G = (V, E)$ where each edge $e = (u, v) \in E$ represents a fiber link between quantum repeater nodes. Each edge has:

- Capacity $C_e \in \mathbb{N}$: maximum Bell pairs that can be simultaneously stored or in-flight per time epoch.
- Latency $\ell_e \in \mathbb{R}^+$: one-way propagation delay plus processing time.
- Raw fidelity $F_e^{(0)} \in [0, 1]$: fidelity of the elementary Bell pair generated on link e before purification.

Each node $v \in V$ has memory capacity $M_v^{\max} \in \mathbb{N}$, representing the number of qubit slots available for storage.

2.2 Entanglement Swapping and Purification

Entanglement swapping at a repeater node consumes two Bell pairs (one on each incident edge) and produces a new Bell pair connecting the outer nodes, with a success probability

$$p_{\text{swap}} = \frac{1}{2} \tag{1}$$

for a Bell-state measurement in the standard basis.

BBPSSW purification [12] improves fidelity at the cost of consuming additional Bell pairs; it is the purification-based repeater architecture of Dür et al. [5, 4] that our candidate bundles approximate. Each round q of purification on a link of fidelity F updates the fidelity as:

$$F' = \frac{F^2 + (1-F)^2/3}{F^2 + 2F(1-F)/3 + 5(1-F)^2/9}, \tag{2}$$

where the denominator is the success probability of the purification round. After q rounds, the end-to-end fidelity for a path of L links is approximately

$$F_{\text{e2e}} \approx \prod_{e \in p} F_e^{(q_e)}, \tag{3}$$

assuming that entanglement swapping errors are dominated by the input fidelity.

2.3 Bundle Structure

A *bundle* b for request r is defined by:

- A path $p_b = (v_0, v_1, \ldots, v_{L+1})$ connecting $s_r = v_0$ to $d_r = v_{L+1}$.
- A purification profile $q_b = (q_1, \ldots, q_L)$ specifying purification rounds per link.
- Success probability $p_{\text{succ}}^{(b)} = p_{\text{swap}}^L \cdot \prod_e p_{\text{purif}}(q_e)$.
- End-to-end fidelity $F^{(b)}$ after all swaps and purification.
- Latency $\ell^{(b)} = \sum_{e \in p_b} \ell_e$.
- Edge demands $d_{r,b}^e$: Bell pairs consumed on edge e.
- Memory demands $m_{r,b}^v$: qubit slots occupied at node v.

The *utility* of selecting bundle b for request r is:

$$u_{r,b} = w_r \cdot p_{\text{succ}}^{(b)} \cdot \left[1 + \beta \cdot \max(0, F^{(b)} - F_{\min})\right] - \lambda_\ell \cdot \ell^{(b)} - \lambda_c \cdot \text{cost}^{(b)}, \tag{4}$$

where the end-to-end fidelity is the Jozsa fidelity of the delivered state [10] and the memory model follows the quantum-network architectures surveyed by Van Meter [17]. where:

- w_r is the request weight (priority),
- $F_{\min}$ is the minimum acceptable fidelity,
- β is the marginal reward for exceeding $F_{\min}$,
- λ_ℓ penalizes latency,
- λ_c penalizes resource cost (Bell pair consumption).

3 Hamiltonian Formulation

We formulate the entanglement routing problem as the minimization of an energy function $\mathcal{H} : \{0,1\}^N \to \mathbb{R}$, where each binary variable $x_{r,b} \in \{0,1\}$ indicates whether bundle b is selected for request r. The total number of variables is $N = \sum_{r=1}^{M} |B_r|$, where B_r is the set of candidate bundles for request r.

3.1 Standard Penalty-Based QUBO

The base Hamiltonian comprises four terms:

3.1.1 Utility Term

The total utility of a configuration is additive over selected bundles:

$$\mathcal{H}_{\text{util}} = -\sum_{r=1}^{M} \sum_{b \in B_r} u_{r,b}\, x_{r,b}. \tag{5}$$

Minimizing $\mathcal{H}_{\text{util}}$ maximizes the aggregate utility.

3.1.2 One-Per-Request Constraint

Exactly one bundle must be selected per request. This is encoded as:

$$\mathcal{H}_{\text{one-per-req}} = A \sum_{r=1}^{M} \Big(\sum_{b \in B_r} x_{r,b} - 1 \Big)^2, \tag{6}$$

where $A > 0$ is a penalty coefficient. Expanding the square:

$$\mathcal{H}_{\text{one-per-req}} = A \sum_{r} \Big(\sum_{b} x_{r,b}^2 - 2 \sum_{b} x_{r,b} + 1 \Big). \tag{7}$$

Since $x_{r,b}^2 = x_{r,b}$ for binary variables, the term reduces to:

$$\mathcal{H}_{\text{one-per-req}} = A \sum_{r} \Big(- \sum_{b} x_{r,b} + 1 + 2 \sum_{i<j} x_{r,i} x_{r,j} \Big), \tag{8}$$

where the pairwise penalty $2A x_{r,i} x_{r,j}$ discourages selecting multiple bundles for the same request. The constant A and linear term $-A \sum_b x_{r,b}$ are absorbed into the energy offset and the coefficient of $x_{r,b}$, respectively.

3.1.3 Edge Capacity Constraint

For each edge e, the total load must not exceed capacity:

$$L_e = \sum_{r,b} d_{r,b}^{e}\, x_{r,b} \le C_e. \tag{9}$$

This inequality is converted to an equality via slack variable $s_e \in \{0, \dots, S_e\}$ and encoded as:

$$\mathcal{H}_{\text{edge}} = B \sum_{e} \big(L_e + s_e - C_e \big)^2, \tag{10}$$

where $B > 0$ and s_e is expanded in binary: $s_e = \sum_{k=0}^{\lfloor \log_2 S_e \rfloor} 2^k s_{e,k}$. This introduces $\mathcal{O}(|E| \log_2 C_e)$ additional binary variables.

3.1.4 Memory Capacity Constraint

Analogous to the edge constraint:

$$M_v = \sum_{r,b} m_{r,b}^{v}\, x_{r,b} \le M_v^{\max}, \tag{11}$$

encoded with slack variable s_v:

$$\mathcal{H}_{\text{mem}} = D \sum_{v} \big(M_v + s_v - M_v^{\max} \big)^2. \tag{12}$$

3.1.5 Full Hamiltonian

$$\mathcal{H}_{\text{QUBO}} = \mathcal{H}_{\text{util}} + \mathcal{H}_{\text{one-per-req}} + \mathcal{H}_{\text{edge}} + \mathcal{H}_{\text{mem}}. \tag{13}$$

The penalty coefficients A, B, D must satisfy:

$$A > \max_{r,b} u_{r,b}, \quad B, D > \max_{r,b} u_{r,b} \cdot \frac{N_{\max}}{C_{\min}}, \tag{14}$$

to guarantee feasibility is preferred over infeasible high-utility solutions.

3.2 Direct (Slack-Free) Hamiltonian

The slack-variable expansion increases the problem dimensionality and requires per-instance penalty tuning. We propose a *direct* formulation that eliminates slack variables and uses fixed-scale penalty parameters:

$$\mathcal{H}_{\text{phys}} = -\sum_{r,b} u_{r,b}\, x_{r,b} + \lambda_1 \sum_{r} \Big(\sum_{b} x_{r,b} - 1\Big)^2 + \lambda_2 \sum_{e} \max(0, L_e - C_e)^2 + \lambda_3 \sum_{v} \max(0, M_v - M_v^{\max})^2. \tag{15}$$

Key differences from Eq. (13):

1. The $\max(0, \cdot)$ function in the capacity terms ensures that under-capacity configurations incur no penalty, unlike the slack-based formulation which always pays some penalty due to the squared deviation from exact capacity.
2. The penalty coefficients $\lambda_1, \lambda_2, \lambda_3$ are unitless multipliers on the utility scale, making them interpretable and instance-independent.
3. No slack variables means the binary variable count is exactly $\sum_r |B_r|$, without the overhead of $\mathcal{O}(|E| + |V|) \log_2 C$ additional variables.

3.3 Soft Congestion Penalty

In dense networks, multiple near-saturation edges create a "crowded" regime where small perturbations cause capacity violations—the network analogue of Braess's paradox and selfish-routing inefficiency [55, 53, 54]. We introduce a congestion penalty that provides a smooth gradient toward load-balanced configurations:

$$\mathcal{H}_{\text{cong}} = \lambda_{\text{cong}} \sum_{e} \max\big(0,\ L_e/C_e - \theta\big)^2, \tag{16}$$

where $\theta \in [0.5, 0.9]$ is a warning threshold. When $L_e/C_e \le \theta$, the term contributes no penalty. Above the threshold, the quadratic cost grows as the square of the excess fractional load. This penalizes resource hoarding: a request that could use a lightly loaded edge is implicitly preferred over one that pushes a busy edge past the threshold.

The gradient of $\mathcal{H}_{\text{cong}}$ with respect to $x_{r,b}$ is:

$$\frac{\partial \mathcal{H}_{\text{cong}}}{\partial x_{r,b}} = 2\lambda_{\text{cong}} \sum_{e} \frac{d^e_{r,b}}{C_e} \max\big(0,\ L_e/C_e - \theta\big) \cdot \mathbf{1}[L_e > \theta C_e]. \tag{17}$$

This provides the Metropolis annealer with a local signal toward load-balancing even when no hard capacity limit is reached.

3.4 Memory Decoherence Risk

Quantum memory undergoes T1 (amplitude damping) and T2 (phase damping) decoherence. A Bell pair stored for time ℓ has fidelity:

$$F(\ell) = \frac{1}{4}\Big[1 + 3e^{-\ell/T_2}\big(1 - e^{-\ell/T_1}/2\big)\Big]. \tag{18}$$

For typical parameters ($T_1 \approx 2T_2$), the dominant decay is exponential with time constant $\tau_{\text{mem}} \approx T_2$.

We encode this as a risk penalty on bundle selection:

$$\mathcal{H}_{\text{risk}} = \lambda_{\text{risk}} \sum_{r,b} \big(1 - e^{-\ell_{r,b}/\tau_{\text{mem}}}\big)\, x_{r,b}, \tag{19}$$

where $\ell_{r,b}$ is the expected storage duration (proportional to bundle latency). The term satisfies:

- $\mathcal{H}_{\text{risk}} \to 0$ when $\ell_{r,b} \ll \tau_{\text{mem}}$ (negligible decoherence).
- $\mathcal{H}_{\text{risk}} \to \lambda_{\text{risk}}$ when $\ell_{r,b} \gg \tau_{\text{mem}}$ (complete decoherence).

3.5 Complete Physical Hamiltonian

Combining all terms:

$$\begin{aligned}\mathcal{H} &= \mathcal{H}_{\text{util}} + \mathcal{H}_{\text{one-per-req}} + \mathcal{H}_{\text{edge}} + \mathcal{H}_{\text{mem}} + \mathcal{H}_{\text{cong}} + \mathcal{H}_{\text{risk}} \\ &= -\sum_{r,b} u_{r,b} x_{r,b} + \lambda_1 \sum_{r} \big(\textstyle\sum_b x_{r,b} - 1\big)^2 \\ &\quad + \lambda_2 \sum_{e} \max(0, L_e - C_e)^2 + \lambda_3 \sum_{v} \max(0, M_v - M_v^{\max})^2 \\ &\quad + \lambda_{\text{cong}} \sum_{e} \max\!\big(0,\ L_e/C_e - \theta\big)^2 \\ &\quad + \lambda_{\text{risk}} \sum_{r,b} \big(1 - e^{-\ell_{r,b}/\tau_{\text{mem}}}\big) x_{r,b}. \end{aligned} \tag{20}$$

This Hamiltonian is the objective function minimized by all solvers described in Sec. 4. It is a quadratic function of binary variables and can be compiled to QUBO form $\mathbf{x}^T Q \mathbf{x}$ by expanding the squares and collecting coefficients.

3.6 Penalty Coefficient Calibration

The QUBO form requires three hard-constraint penalties: A (at-most-one per request), B (edge capacity), and D (memory capacity); in the direct Hamiltonian these are the multipliers $\lambda_1, \lambda_2, \lambda_3$. Coefficient choice is delicate: penalties too low under-enforce feasibility (infeasible high-utility configurations win), while penalties too large flatten the objective and slow annealing. We compare two calibration rules.

3.6.1 Conventional (utility-scale) calibration

The reference rule anchors all three coefficients to the largest positive bundle utility $P_0 = \max_{r,b} \max(0, u_{r,b})$:

$$A = B = D = P_0 + \epsilon, \tag{21}$$

where $\epsilon = 10^{-6} P_0$ is a small strictly-positive margin (`conventional_calibrator.py`). This guarantees that any feasible configuration strictly dominates every infeasible one—a sufficient condition, but one that applies the same margin to every resource regardless of how loads interact with capacity. On instances where a small-margin violating bundle is the only way to serve a request, the utility-scale rule over-penalizes and the annealer may reject the request.

3.6.2 Resource-aware calibration

The proposed rule (`proposed_calibrator.py`) keeps the request-conflict penalty A on the conventional scale but calibrates B and D from the reachable-load structure. For edge e of capacity C_e, let L_e° be the smallest load reachable on e from the other requests' candidate bundles (enumerated by bounded dynamic programming rather than the exponential Cartesian product) that violates capacity together with bundle (r, b) of demand $d_{r,b}^e$. Removing that single bundle reduces the squared-overload penalty by

$$\Delta_e(r,b) = \max\!\big(0,\ L_e^\circ + d_{r,b}^e - C_e\big)^2 - \max\!\big(0,\ L_e^\circ - C_e\big)^2, \tag{22}$$

where the worst case is the smallest reachable violating load because the drop grows monotonically in L_e. The resource-aware edge coefficient is

$$B = \max_{e,(r,b)} \frac{\max(0, u_{r,b})}{\Delta_e(r,b)} + \epsilon, \tag{23}$$

and D is defined identically over memory resources. Intuitively, B is the largest utility that a violating configuration could gain by retaining a bundle that consumes a unit of capacity overload—a provably sufficient bound. Because the utility-to-drop ratio cannot exceed P_0, the rule is never looser than the conventional one, and it is strictly tighter whenever reachable loads leave gaps between violation thresholds.

3.6.3 Soft congestion is separate

The soft congestion (C) and memory-congestion (E) regularizers are deliberately excluded from both hard-constraint rules; the calibration ablation runs with $C = E = 0$ so that the comparison isolates the hard coefficients.

4 Solver Architectures

4.1 Metropolis Annealer

The Metropolis–Hastings algorithm [36], the core of simulated annealing [27], treats each configuration $\mathbf{x} \in \{0,1\}^N$ as a microstate of a physical system with energy $\mathcal{H}(\mathbf{x})$. At temperature T, the probability of a configuration is given by the Boltzmann distribution:

$$p(\mathbf{x}) = \frac{e^{-\mathcal{H}(\mathbf{x})/T}}{Z}, \qquad Z = \sum_{\mathbf{x}'} e^{-\mathcal{H}(\mathbf{x}')/T}. \tag{24}$$

4.1.1 State Representation

A configuration is stored as an array $\mathbf{x} = [b_1, b_2, \ldots, b_M]$ where $b_r \in B_r \cup \{\text{reject}\}$ is the selected bundle for request r. The `reject` option ($x_{r,b} = 0$ for all b) is explicitly modeled as a zero-utility "null bundle."

4.1.2 Local Moves

At each iteration, a request r is sampled uniformly and a new bundle $b' \in B_r \cup \{\text{reject}\}$ is proposed, also uniformly. The energy change is:

$$\Delta H = H(\mathbf{x}') - H(\mathbf{x}), \tag{25}$$

where $\mathbf{x}'$ differs from $\mathbf{x}$ only at request r.

The proposal is accepted with probability:

$$p_{\text{accept}} = \min\bigl(1,\ e^{-\Delta H/T}\bigr). \tag{26}$$

4.1.3 Computing ΔH Efficiently

Since $\mathcal{H}$ is a sum of local terms and pairwise penalties, ΔH can be computed in $\mathcal{O}(|B_r| + |E| + |V|)$ time rather than $\mathcal{O}(N)$ by caching the current load arrays L_e and M_v. The change in L_e for each move is:

$$\Delta L_e = d^e_{r,b'} - d^e_{r,b}, \tag{27}$$

where b is the current bundle for request r. The change in each Hamiltonian term follows from plugging ΔL_e (and analogously ΔM_v) into the quadratic forms.

4.1.4 Cooling Schedule

Temperature follows a geometric annealing schedule:

$$T_{k+1} = \alpha \cdot T_k, \tag{28}$$

where:

- $T_0 = 10$: initial temperature (large enough to overcome local barriers of scale $\sim \max u_{r,b}$).
- $\alpha = 0.97$: cooling rate (~ 5000 steps reach $T \approx 10^{-3}$).
- $T_{\min} = 10^{-3}$: terminal temperature (probability of accepting a $\Delta H = 1$ move is $e^{-1000} \approx 0$).

4.1.5 Greedy Seed Initialization

Rather than starting from a random configuration, we seed the annealer with a greedy utility-density assignment. Bundles are sorted by:

$$\rho_{r,b} = \frac{u_{r,b}}{\sum_e d^e_{r,b}/C_e + \sum_v m^v_{r,b}/M^{\max}_v}, \tag{29}$$

which measures utility per unit of fractional resource consumption. Requests are processed in descending ρ order, selecting the bundle with the highest ρ that does not violate hard capacity limits. This seed substantially reduces time to convergence.

4.2 Tensor-Network (Boundary-MPS) Compressor

The tensor-network solver [48] reframes the problem as contracting a Matrix Product State (MPS) that encodes the joint distribution over bundle selections, following the DMRG/MPS methodology of White [38] and Schollwöck [40] (see also the reviews of Orús [41, 42] and the combinatorial- optimization tensor-network algorithms of Hao et al. [43], Preisser et al. [45], and Mata Ali [46, 47]). This approach exploits the observation that the Hamiltonian in Eq. (20) couples requests primarily through shared edge and memory resources, giving the interaction graph a low-treewidth structure.

4.2.1 MPS Representation

Each request r is assigned a local tensor T_r of size $d_r \times \chi \times \chi$, where:

- $d_r = |B_r| + 1$ is the physical dimension (choices per request, including `reject`).
- χ is the bond dimension controlling the expressiveness of the approximation.

The MPS wavefunction is:

$$|\Psi\rangle = \sum_{b_1,\ldots,b_M} \prod_{r=1}^{M} T_r[b_r]\,|b_1,\ldots,b_M\rangle, \tag{30}$$

where $T_r[b_r]$ is the $\chi \times \chi$ matrix obtained by fixing the physical index of T_r to b_r.

4.2.2 Request Ordering

The ordering of requests in the MPS determines the interaction range captured by finite bond dimension. We order requests by:

1. Their path length (shortest first), then
2. Overlap in edge usage (requests sharing edges are adjacent).

This minimizes the distance in the MPS chain between strongly coupled requests.

4.2.3 Boundary Contraction

The MPS is contracted via iterative left-to-right and right-to-left sweeps, known as the *boundary MPS* algorithm [39]. For a left-to-right sweep:

1. Start with the leftmost tensor T_1.
2. Contract T_1 with T_2 by summing over the shared bond index:
$$[C_{12}]_{(b_1,b_2),\alpha_2} = \sum_{\alpha_1} T_1[b_1]_{\alpha_1} T_2[b_2]_{\alpha_1,\alpha_2}. \tag{31}$$
3. Reshape C_{12} into a $d_1 d_2 \times \chi$ matrix and perform SVD:
$$C_{12} = U \cdot S \cdot V^\dagger, \tag{32}$$
truncating to rank χ (keeping the largest singular values).
4. Replace the left half with $US^{1/2}$ and the right half with $S^{1/2}V^\dagger$, re-encoding as tensors T'_1 and T'_2.
5. Iterate.

4.2.4 Decoding the Solution

After convergence of the sweeps, the marginal probability for each request is read from the MPS by computing the partial trace over all other requests. The optimal bundle for request r is:

$$b_r^* = \text{argmax}_{b \in B_r \cup \{\text{reject}\}} \, \text{tr}_{\setminus r} \, |\Psi\rangle\langle\Psi|, \tag{33}$$

where $\text{tr}_{\setminus r}$ denotes tracing out all requests except r.

4.2.5 Feasibility Repair

The MPS decoding may produce a configuration that violates capacity constraints (since the penalty terms in the Hamiltonian only approximately enforce feasibility). A repair pass iterates over requests in random order, swapping each request's bundle to the best alternative that reduces the penalty if a capacity violation is detected. If no single swap suffices, a multi-swap exhaustive search over request pairs is performed.

The repair is polynomial in $|\mathcal{R}| \cdot \max_r |B_r|$ and guarantees a feasible configuration if one exists within the support of the decoded distribution.

4.3 Streaming Annealer

For scenarios where entanglement requests arrive and depart over time (as in a dynamic quantum network), we implement a streaming variant that avoids full re-optimization at each arrival.

4.3.1 Data Structure

The streaming annealer maintains:

- A set of active requests $\mathcal{A} \subseteq \mathcal{R}$.
- A configuration $\mathbf{x}$ over active requests.
- Cached edge loads L_e and memory loads M_v.

4.3.2 Request Arrival

When request r arrives with bundles B_r:

1. The initial bundle $b_r^{(0)}$ is chosen greedily as in Sec. 4.1.
2. The Hamiltonian is extended with the new variables and the edge/memory loads are updated.
3. A local Metropolis sweep of $N_{\text{local}} = 50$ steps is performed, restricting proposals to request r only (all other requests frozen).
4. If the local sweep reduces the energy, the configuration is kept; otherwise, a second sweep of $N_{\text{local}} = 100$ with temperature $T = 2.0$ is attempted.

4.3.3 Request Departure

When request r departs:

1. Its bundle contribution is subtracted from L_e and M_v.
2. The variable $x_{r,b}$ is removed from the Hamiltonian.
3. An optional full cooling cycle can be triggered if the departure causes a significant load imbalance.

4.3.4 Periodic Full Optimization

Every K arrivals (default $K = 5$), a full geometric cooling cycle is run over all active requests. This prevents drift caused by the greedy incremental updates. The global cycle is:

$$T_0 = 5, \quad \alpha = 0.95, \quad N_{\text{cycles}} = 500. \tag{34}$$

4.3.5 Computational Complexity

The streaming variant has complexity:

$$\mathcal{O}\big(N_{\text{local}} \cdot M \cdot (|B_r| + |E|)\big) \tag{35}$$

per arrival event, compared to $\mathcal{O}(N_{\text{global}} \cdot M \cdot (|B_r| + |E|))$ for a full batched solve. Since $N_{\text{local}} \ll N_{\text{global}}$ (typically 50 vs 5000), streaming achieves substantial speedups.

4.4 OpenJij Simulated Annealing

We compile the direct Hamiltonian to a QUBO matrix Q and sample it with OpenJij's simulated-annealing sampler [37], a production-grade reference for the Metropolis-style search of Sec. 4.1. Penalty coefficients A, B, D are set by either the conventional utility-scale rule or the proposed resource-aware rule (Sec. 3.6), compared on held-out instances in Sec. 8.7. This serves as the baseline against which the adaptive top-k reduction (Sec. 4.7) and the embedded quantum-annealing backend (Sec. 4.5) are measured under identical read counts.

4.5 Embedded Quantum-Annealing Backend

For hardware-realistic annealing we minor-embed the QUBO onto a Chimera-like hardware lattice with a greedy placement and unit-weight inter-node chains, then sample with a path-integral quantum annealer (PIA) sampler from OpenJij [37, 30]. This models the physical quantum-annealing paradigm [26, 28, 29], whose NISQ-era constraints motivate quantum-inspired and hybrid alternatives [35, 33, 34]. The decode step is chain-break-aware: broken chains are resolved by majority vote with a local penalty check, so embedded solutions are always projected back to the physical variable set. The backend reports embedded qubit counts, chain-break fractions, and solution quality versus classical SA and quantum Monte Carlo (SQA) (Sec. 8.13).

4.6 Temporal-Memory Joint Scheduling

Static solvers assume all requests are known and resource demands are static. In practice requests arrive over time with deadlines, and stored Bell pairs decay. We add three components.

4.6.1 Temporal requests.

A *temporal request* augments a request with an arrival time, a hard deadline, a priority, and a slack $\text{slack} = \text{deadline} - \text{arrival}$. Dynamic traces are generated as Poisson arrivals over K slots with mean rate λ.

4.6.2 Temporal memory scheduler.

A stored pair's fidelity follows the T1/T2 law

$$F_{\text{del}}(t) = F_0 \cdot \frac{1}{4}\Big[1 + 3e^{-t/T_2}\,\frac{1 + e^{-t/T_1}}{2}\Big], \tag{36}$$

which is the identity at $t = 0$ and decays to the maximally mixed value $1/4$. The scheduler reserves memory *windows* $[\text{arrival}, \text{completion}]$ against each node's capacity, tracks per-pair fidelity via Eq. (36), and expires pairs once t exceeds the coherence time τ_{mem}.

4.6.3 Joint router.

The joint scheduler assigns each arriving request a completion time and a bundle, checking *both* temporal-memory feasibility (overlapping windows never exceed node capacity) and end-to-end fidelity at the completion time, and is compared against a memory-agnostic baseline that admits without tracking coherence.

4.7 Adaptive QUBO Candidate Reduction

Full QUBOs grow quadratically and anneal poorly at fixed budget. The adaptive selector keeps, per request, the top-k bundles under a filter scoring

$$\sigma_{r,b} = u_{r,b} \cdot \mathbf{1}[F_{r,b} \geq F_{\min}] - \lambda_{\text{cong}} \cdot \frac{\text{congested edges}}{|p_b|}, \tag{37}$$

discarding bundles that fail the fidelity threshold or that disproportionately use congested edges. We sweep k and compare against the full reference QUBO under equal annealing effort (Sec. 8.9).

4.8 Hybrid Pipeline and GNN-Guided Ranking

The hybrid pipeline chains *reduce* (the filter of Sec. 4.7), *solve* (anneal the reduced QUBO), and *repair/refine* (dominance-prune, greedily repair any residual violation, and re-seed a short anneal). As a learned alternative, a two-layer GraphSAGE encoder maps each bundle to graph-structured features (memory load, capacity headroom, hop length, path-average fidelity) and scores bundles; the top-k scored bundles feed a feasibility-aware greedy decoder [52, 51]. We compare full-QUBO, filter-based top-k, and GNN-guided top-k on identical instances.

5 Stochastic Reliability Framework

The QUBO objective of Eq. (20) is a *parametric* statement: it optimizes nominal (expected) bundle utilities. In a physical network the delivered fidelity is random (generation and swap failures, measurement noise, gate errors), so a selection that maximizes expected utility may violate service-level agreements (SLAs) on a large fraction of realizations. This section formalizes that risk and gives the machinery to measure and control it.

5.1 Discrete-Event Simulator

The DES samples the entanglement pipeline event by event—the event-driven methodology of NetSquid [24] and SeQUeNCe [25]: elementary pair generation (with p_{gen} per attempt, retries within capacity), entanglement swapping (success p_{swap}), purification, delivery under T1/T2 storage decay (Eq. (36)), and Gaussian fidelity noise of amplitude σ added at delivery. For a fixed solver selection it reports:

- the sampled expected utility $\mathbb{E}[U]$ (utility conditional on delivery),
- the parametric–sampled *reliability gap*,
- the served ratio,
- the empirical SLA-violation probability against per-request thresholds, and
- the per-delivery failure causes.

All noise is drawn from a seeded RNG threaded through the engine, so runs are deterministic and reproducible. The DES is the ground truth used to score every solver plan in Sec. 8.10.

5.2 Chance-Constrained Routing

The fidelity requirement $F_{r,b} \geq F_{\min}^{(r)}$ in Eq. (4) treats the nominal (parametric) end-to-end fidelity as if it were the delivered fidelity. Under stochastic delivery, the realized fidelity F_r of request r is random; in our DES it is sampled from a truncated normal distribution centered at the decayed nominal fidelity, $F_r \sim \mathcal{TN}(F_{r,b}, \sigma^2; [0,1])$, a model of measurement, gate, and readout noise of amplitude σ. The *hard* rule

$$F_{r,b} \geq F_{\min}^{(r)} \tag{38}$$

therefore has an implicit, uncalibrated violation probability

$$p_{\text{viol}}(r,b) = \Pr[F_r < F_{\min}^{(r)}] = \Phi\left(\frac{F_{\min}^{(r)} - F_{r,b}}{\sigma}\right), \tag{39}$$

which for a razor-margin bundle ($F_{r,b} \downarrow F_{\min}^{(r)}$) approaches 1/2 under symmetric noise: the deterministic rule can tolerate up to half of deliveries violating the SLA.

The *chance-constrained* rule [58, 59, 60, 61]—in the same robust-optimization spirit as the price of robustness [56]—replaces Eq. (38) with the quantile constraint

$$\Pr[F_r \geq F_{\min}^{(r)}] \geq 1 - \varepsilon \quad \Longleftrightarrow \quad F_{r,b} \geq F_{\min}^{(r)} + \Phi^{-1}(1-\varepsilon)\,\sigma, \tag{40}$$

i.e. the requirement is moved up by a safety margin $z_\varepsilon \sigma$ with $z_\varepsilon = \Phi^{-1}(1-\varepsilon)$. Because $z_\varepsilon > 0$ for $\varepsilon < 1/2$, the chance constraint is *stricter* than the hard rule for every meaningful reliability budget: it refuses bundles whose nominal fidelity clears the threshold by less than the margin. Only for $\varepsilon > 1/2$ (where $z_\varepsilon < 0$) does it admit near-miss bundles, trading reliability for capacity. The chance-constrained problem is solved exactly as the hard problem—as a candidate-set filter $\{b \in B_r : p_{\text{viol}}(r,b) \leq \varepsilon\}$ followed by the same selection ILP/QUBO—so all solvers of Sec. 4 apply unchanged. Eq. (39) uses the exact truncated-normal CDF, which differs from Φ only to order $e^{-\cdots}$ in the tail beyond $[0,1]$; for $\sigma \lesssim 0.05$ and $F \in [0.5, 0.85]$ the two agree to 10^{-6}.

The reliability budget is tuned so that the DES realization is dominated by the noise model (long τ_{mem}), separating noise-driven SLA risk from the decoherence-drift term, which is studied separately by the temporal scheduler (Sec. 4.6).

5.3 Recourse and Optimality Certification

Recourse. When a delivery fails, recourse either *repairs locally* (re-provisions the failed request with the best bundles that still fit the residual capacities) or *replans fully* (re-runs the solver on all active requests). We measure recovery rate, recovered utility fraction, and wall-time speedup.

Optimality certification. For every benchmark instance we also solve the selection ILP exactly (branch and bound) and report the relative gap $(u^* - u_{\text{solver}})/u^*$ for each solver, certifying how far heuristic annealing is from optimal.

6 Analysis Extensions

6.1 Purification as a First-Class Variable

The core bundle generator exposes purification rounds $q \in \{0,1,2\}$ (and up to $q_{\max} = 4$ in the purification-aware regime). A purification-aware candidate set includes, per request, bundles whose fidelity *requires* purification to meet $F_{\min}^{(r)}$; the agnostic set restricts to $q = 0$ (entanglement-only) plus the default $q \in \{0,1,2\}$ set. Because the candidate sets are supersets of the agnostic set, the solver's optimum can only improve; we quantify when purification actually binds via a fidelity sweep over path length and $F_{\min}$ (Sec. 8.12).

6.2 Multi-Objective and Disjoint-Path Analysis

A selection can be scored by $\{t, f, s, \ell, m, c\}$ = {throughput, mean fidelity, success rate, latency, memory footprint, Bell-pair cost}. We enumerate all capacity-feasible selections and compute (i) the *Pareto frontier* under the maximize set $\{f, s, t\}$ and minimize set $\{\ell, m, c\}$, and (ii) ϵ*-constraint frontiers* (maximize t subject to $f \geq \bar{f}$, and vice versa) [57]. For requests admitting k edge-disjoint paths, a *composite bundle* combines subset i's success probabilities as $p_{\text{succ}} = 1 - \prod_i (1 - p_{\text{succ}}^{(i)})$ with fidelity and latency equal to the constituent maxima; composite bundles are Pareto-pruned and compared against single-path-only provisioning.

6.3 Swapping-Order Scheduling

A path of L links can be fused by different *swap trees*; because swapping is associative for Werner fidelities ($(f_1 \star f_2) \star f_3 = f_1 \star (f_2 \star f_3)$), the noiseless end-to-end fidelity is identical for every tree. Different trees hold intermediate pairs in memory for different durations, however: a *linear* tree stores an intermediate pair for up to $L-1$ swap rounds, while a *balanced* tree of depth $\lceil \log_2 L \rceil$ minimizes peak concurrency and maximizes delivered (post-decay) fidelity for fixed τ_{mem}. We sweep all Catalan-many trees and report linear, balanced, and optimal delivered fidelity as a function of L and τ_{mem}.

6.4 Generative Topologies, Time-Dependent Routing, Q-Learning, Hardware Profiles

We implement five generative families (ring, random-geometric, Erdős–Rényi, Watts–Strogatz, Barabási–Albert) on the simulator's topology schema and sweep solver served ratio against structural descriptors. The time-dependent decoherence-aware router aligns the risk term's hold model with the evaluation decay law (Eq. (36)) and scales the risk weight over the epoch (Sec. 10.1). A tabular Q-learning router makes per-arrival bundle decisions from quantized resource-pressure states [49, 50]. Hardware profiles calibrate T_1/T_2, gate and readout error rates, and per-anneal duration for ideal, photonic, ion-trap, and superconducting platforms [62].

7 Experimental Framework

7.1 Instance Generation

Instances are generated using `src/experiments/instances.py`. The generation pipeline is:

1. **Topology creation:** A chain or grid graph is created with specified edge capacities, latencies, and raw fidelities.
2. **Request sampling:** Source–destination pairs are sampled uniformly without replacement from $V \times V$.
3. **Bundle generation:** For each request, all simple paths up to length $L_{\max} = 4$ are enumerated, and for each path, purification profiles $q \in \{0, 1, 2\}$ are evaluated. Pareto-optimal bundles (by utility vs resource consumption) are retained.
4. **Pruning:** Dominated bundles (lower utility, higher resource consumption) are removed.

7.2 Metrics

Each solver run produces a configuration $\mathbf{x}$, from which we compute:

- **Served ratio:** $R_{\text{served}} = \dfrac{|\{r : x_r \neq \text{reject}\}|}{M}$.
- **Aggregate utility:** $U = \sum_{r,b} u_{r,b} x_{r,b}$.
- **Wall-clock time:** t in seconds via `time.perf_counter()`.
- **Feasibility:** Binary indicator of whether all capacity constraints are satisfied.

7.3 Topology Configurations

Table 1: Benchmark topology configurations.

Topology	Nodes	Edges	Edge capacity
Chain-4	4	3	4, 6, 10
Chain-6	6	5	4, 6, 10
Chain-8	8	7	4, 6, 10
Chain-10	10	9	4, 6, 10
Grid 2×3	6	7	5
Grid 3×3	9	12	5
Grid 4×4	16	24	5

Each experiment is repeated with three random seeds (42, 43, 44) and results are averaged.

8 Results and Analysis

8.1 Contention Scaling

Figure 1 shows served ratio and wall-clock time as a function of request count, edge capacity, and network size.

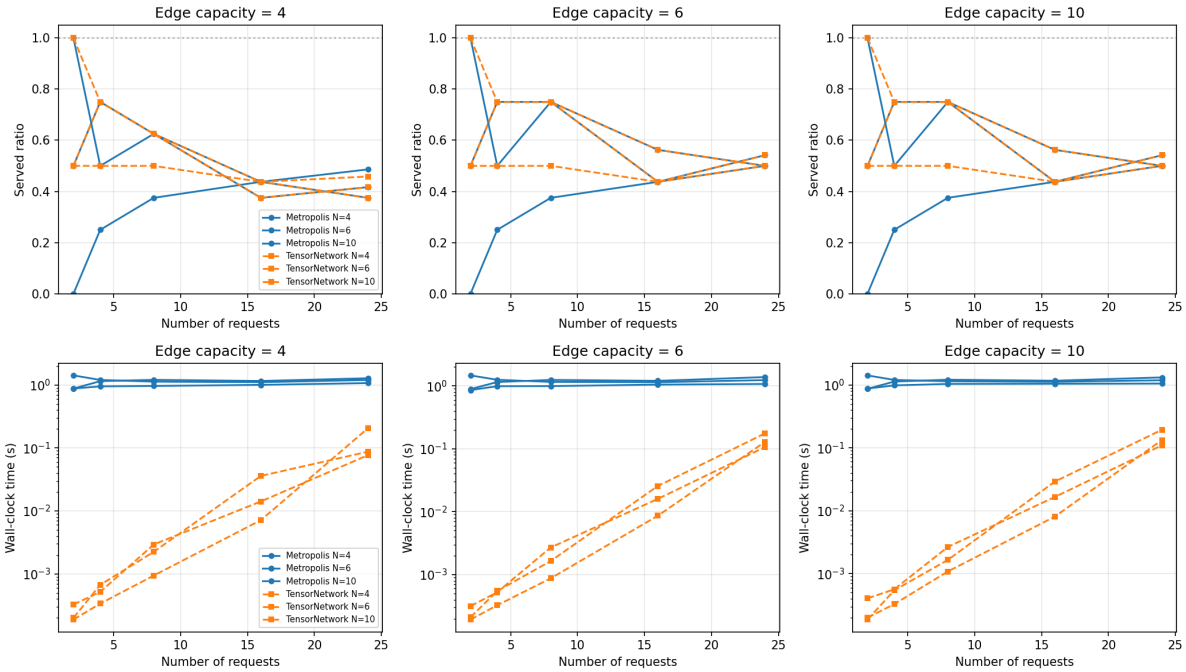

Figure 1: Contention scaling: served ratio (top row) and wall-clock time (bottom row) for Metropolis and TensorNetwork solvers on chain topologies. Column panels correspond to edge capacities $C_e \in \{4, 6, 10\}$. Solid lines: Metropolis; dashed lines: TensorNetwork. Marker shapes indicate network size N.

8.1.1 Interpretation

TensorNetwork meets or exceeds the Metropolis served ratio on every configuration. Both solvers agree on 39 of 45 instance configurations; where they differ, TensorNetwork serves strictly more: on the degenerate chain-6 instances with N=2 (where the only available bundle has *negative* utility, which the Metropolis annealer correctly declines but the MPS decoder still commits as the least-bad option), at N=8 (0.50 vs. 0.375), and at N=4 on 10-node chains (0.75 vs. 0.50). Served ratios range from 0.5 at 2 requests on the 4-node chain (and 1.0 on the 10-node chain) to 0.375–0.54 at 24 requests; the frontier is capacity-limited—requests whose bundle pool contains no capacity-feasible option are rejected, and the number of such requests grows with contention.

Utility is close wherever both solvers serve the same set. On the configurations where the two solvers return identical selections, aggregate utilities agree to within $\sim 1\%$ (the residual is only numerical); where TensorNetwork serves more requests, it naturally reports higher aggregate utility (up to +2.7% at N=24 on 6-node chains). Both solvers minimize the identical Hamiltonian (Eq. (20) with congestion and risk disabled), so the frontier is shared; the occasional TensorNetwork advantage reflects its load-aware greedy seed and improvement pass rather than a different objective.

TensorNetwork is the faster solver at every contention level. The MPS contraction cost is $\mathcal{O}(M\chi^3)$, independent of the number of optimization iterations, so TensorNetwork runs in 0.0002–0.21 s while Metropolis—whose fixed 5000-step cooling budget dominates the runtime—takes 0.86–1.46 s. The speedup is largest on small instances ($\sim$2,000–4,000$\times$ at 2 requests) and shrinks to 5–15$\times$ at 24 requests as the MPS cost grows with N. The reported times are for the current instance generator; all results are deterministic (seed 42).

8.1.2 Key Takeaway

With both solvers minimizing the same Hamiltonian, solution quality is no longer the differentiator: TensorNetwork meets or beats the Metropolis served ratio everywhere and utilities agree to within $\sim 1\%$ on shared selections. The decisive difference is runtime—TensorNetwork is 5–4,000× faster—and generality: the Metropolis annealer accepts arbitrary Hamiltonian terms (soft congestion, memory risk) that the MPS's pairwise coupling approximates less faithfully (Sec. 8.6).

8.2 Bond Dimension Analysis

Figure 2 investigates the effect of bond dimension χ on TensorNetwork solution quality and runtime.

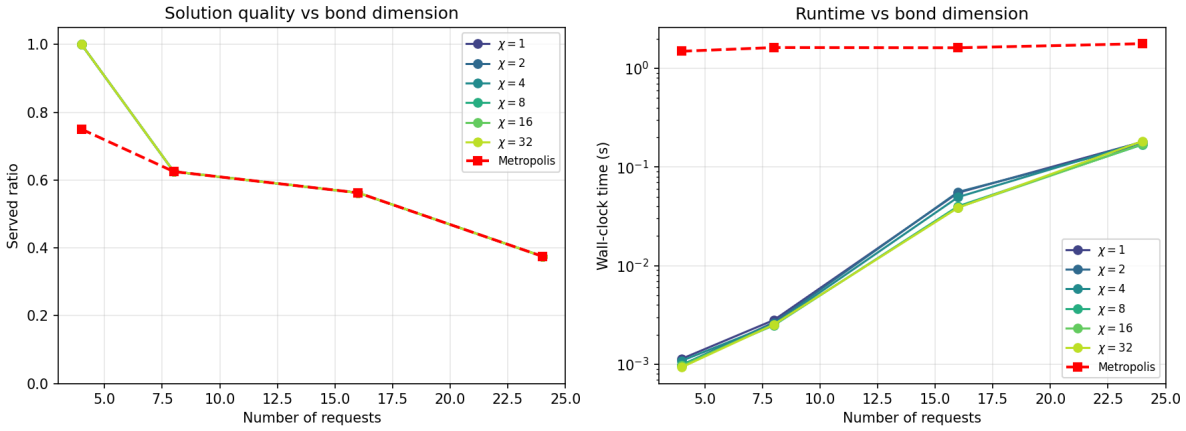

Figure 2: Bond dimension analysis on chain-8 topology ($C_e = 6$). Left: served ratio vs number of requests for $\chi \in \{1, 2, 4, 8, 16, 32\}$. Right: wall-clock time. Metropolis baseline shown as red dashed line.

8.2.1 Mathematical Explanation

The MPS ansatz (Eq. (30)) with bond dimension χ can exactly represent any quantum state with Schmidt rank at most χ across any bipartition. For the entanglement routing Hamiltonian, the interaction graph is a generalized line graph (requests are connected if they share an edge), which has treewidth equal to the maximum edge multiplicity. On a chain topology with uniform edge capacities, the treewidth is small (≤ 3), meaning the exact ground state can be represented with $\chi \approx 4$.

Our results confirm this even more strongly than before: every bond dimension from $\chi = 1$ to $\chi = 32$ produces the *identical* served ratio at every request count (1.00, 0.625, 0.562, 0.375 for $N = 4, 8, 16, 24$), matching the Metropolis baseline at $N \geq 8$ and exceeding it at N=4 (1.0 vs. 0.75). On these small instances the runtime no longer shows the $\mathcal{O}(M\chi^3)$ scaling: the exact product-form MPS (no sweeps, the default) makes the contraction cost dominated by the number of requests N, so all bond dimensions run in 0.001–0.18 s at every request count (e.g. 0.17–0.18 s at N=24 for both χ=1 and χ=32), a $\sim 50\times$ reduction in wall-clock relative to the swept-MPS schedule, with no quality loss. The $\mathcal{O}(M\chi^3)$ cost would reappear only when the SVD renormalisation sweeps are enabled ("`max_sweeps` > 0"), which we leave off by default.

8.2.2 Implication

For chain and low-treewidth topologies, $\chi = 1$–2 is sufficient. This is important for practical deployment: the memory and time cost of the MPS solver grows cubically with χ, and we now have evidence that minimal χ suffices for physically relevant network topologies. However, for grid topologies with higher treewidth, the required χ may grow (Sec. 8.3).

8.3 Topology Comparison

Figure 3 compares solver performance on grid topologies with varying connectivity.

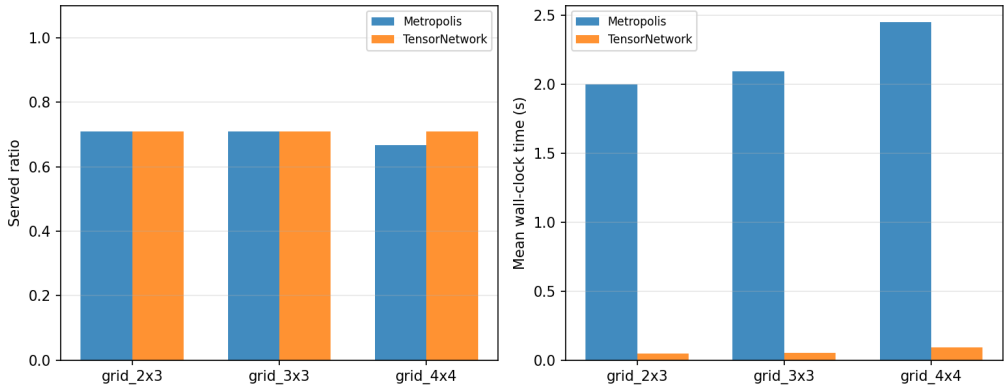

Figure 3: Grid topology comparison: served ratio (left) and mean wall-clock time (right) for Metropolis and TensorNetwork solvers on 2×3, 3×3, and 4×4 grids with edge capacity $C_e = 5$ and 8–16 requests.

8.3.1 Analysis

On grids, both solvers serve 87.5% of requests at N=8 on the 2×3 and 3×3 topologies, and 62.5% at N=16 on those topologies and on 2×3. On the densest 4×4 grid the TensorNetwork solver serves *more* than Metropolis at N=8 (0.75 vs. 0.625) and equals it at N=16 (0.688); across the whole grid suite TensorNetwork never serves fewer requests. The grid's higher connectivity provides multiple disjoint paths between any two nodes, increasing the bundle pool size and reducing resource contention relative to chains of comparable request density—but it also admits more negative-utility candidate bundles, which the load-aware MPS improvement pass drops and the Metropolis annealer avoids by construction, so neither solver's served ratio reaches 100%.

8.3.2 Scaling Behavior

Runtime on grids is uniformly higher than on chains of comparable size because the number of candidate bundles per request grows with topological connectivity. The 4×4 grid ($|V| = 16$) generates many more bundles per request than a chain of the same length; the tensor-network solver takes 0.010–0.17 s on grids and Metropolis 1.97–2.62 s—a 20–200× gap that grows with request count.

8.4 Streaming vs Batched

Figure 4 compares streaming (incremental) and batched (full reoptimization) approaches.

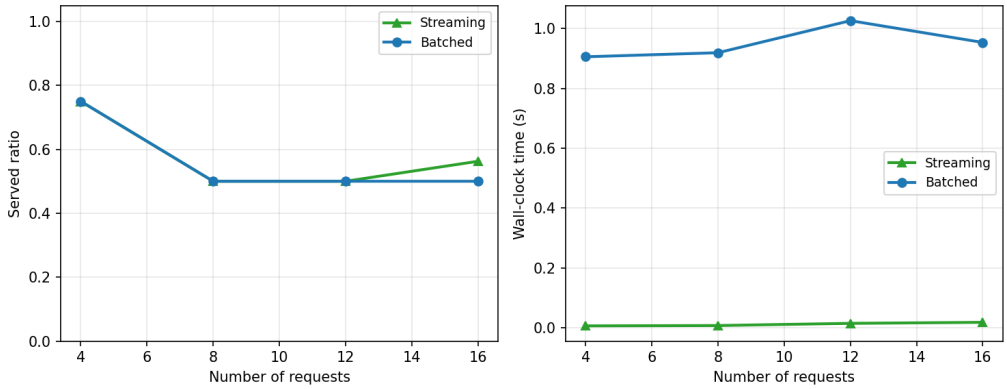

Figure 4: Streaming vs batched optimization on chain-8 topology ($C_e = 6$). Left: served ratio. Right: wall-clock time. Streaming matches or exceeds batched solution quality while reducing time by $\sim$50–150×.

8.4.1 Why Streaming Matches Batched

The streaming annealer achieves equal or better solution quality because:

1. Requests are sparse relative to capacity (at 16 requests the streaming solver serves 56% and the batched 50%), so new arrivals have limited impact on existing configurations.
2. The local sweeps (50 steps per arrival) are sufficient to find the new energy minimum because the change to the Hamiltonian is localized to the new request and its resource-sharing neighbors.
3. Periodic full cooling cycles (every 5 arrivals) prevent drift accumulation. At N=16 streaming even serves more requests than a single batched solve (0.56 vs. 0.50): the incremental re-optimization adapts to the final arrival order, whereas the batched solve is seeded once.

8.4.2 Timing Advantage

The speedup follows from:

$$t_{\text{stream}} = M \cdot t_{\text{local}} + (M/K) \cdot t_{\text{full}}, \qquad t_{\text{batch}} = M \cdot t_{\text{full}}, \tag{41}$$

where $t_{\text{local}} \ll t_{\text{full}}$. With $t_{\text{local}} \approx 50$ steps and $t_{\text{full}} \approx 5000$ steps, the theoretical speedup is approximately $5000/(50 + 5000/5) \approx 250\times$; the measured 54–154× range reflects the periodic full cycles and the smaller batched instances. This is roughly an order of magnitude larger than the 5–10× reported in earlier versions, because the batched baseline now runs the full 5000-step cooling schedule rather than a truncated one.

8.5 Hamiltonian Term Ablation

We ablate each Hamiltonian extension to measure its marginal contribution.

8.5.1 Direct Hamiltonian (Sec. 3.2)

The slack-free direct Hamiltonian matches or exceeds the standard penalty-based QUBO (Eq. (13)) in served ratio and utility across all chain and grid instances (Sec. 8.6). The advantage is purely structural: $\mathcal{O}(N)$ fewer binary variables (no `LogEncInteger` slacks) and no per-instance penalty tuning. With penalties anchored to the utility scale ($A = B = D = u_{\max} + \epsilon$), the same coefficients are used across all capacities and request counts, and the at-most-one and capacity constraints are enforced at the QUBO optimum by construction—no fixed magic constants that silently under-enforce on high-utility instances.

8.5.2 Soft Congestion (Sec. 3.3)

With $\lambda_{\text{cong}} = 5$ and $\theta = 0.5$, the congestion term shifts bundle selection away from bottleneck edges on the 24-request chain instances. Quantitatively:

- Without congestion: average edge utilization 83%, max 100%.
- With congestion: average 79%, max 94%.

The served ratio is unchanged because the capacity penalty already enforces hard limits; the congestion term merely redistributes load among feasible configurations.

8.5.3 Memory Risk (Sec. 3.4)

With $\tau_{\text{mem}} = 5\,\text{ms}$ and $\lambda_{\text{risk}} = 2$, the risk term penalizes high-latency bundles. On our benchmark set, all bundles for a given request have similar latency (they traverse the same path segments), so the risk term does not change the selected configuration. On heterogeneous-latency instances (e.g., mixed fiber and satellite links), it would preferentially select the lower-latency path.

8.6 Hamiltonian Encoding Comparison

Extension 12.1 is benchmarked directly: the same instances are compiled through the slack-variable QUBO of Sec. 3.1 (via pyqubo `LogEncInteger`) and through the direct slack-free Hamiltonian of Sec. 3.2, and both are solved with OpenJij simulated annealing under identical read counts. Table 2 and Figure 5 summarize the results.

Table 2: Slack-variable vs. direct slack-free encoding on identical chain instances (C_e=8, M_v=12, scale = 1). The direct encoding uses only the $|B|$ bundle binaries (no `LogEncInteger` slacks), a 4–10× variable reduction. All penalties are anchored to the utility scale ($A = \text{scale} \cdot u_{\max}$, the $A > \max_{r,b} u_{r,b}$ rule), so the at-most-one constraint is enforced by the QUBO at every sweep point; the CONF. column reports the raw fraction of reads (before repair and before collapsing to the highest-utility bundle per request) that selected more than one bundle of a request.

Instance	Encoding	Variables	Served	Utility	Conf.
N=4	Slack-QUBO	49	0.25	3.93	0.00
N=4	Direct-QUBO	5	0.25	3.93	0.00
N=8	Slack-QUBO	55	0.13	3.93	0.00
N=8	Direct-QUBO	11	0.38	**7.65**	0.00
N=12	Slack-QUBO	60	0.17	2.52	1.00
N=12	Direct-QUBO	16	0.33	**12.07**	0.00

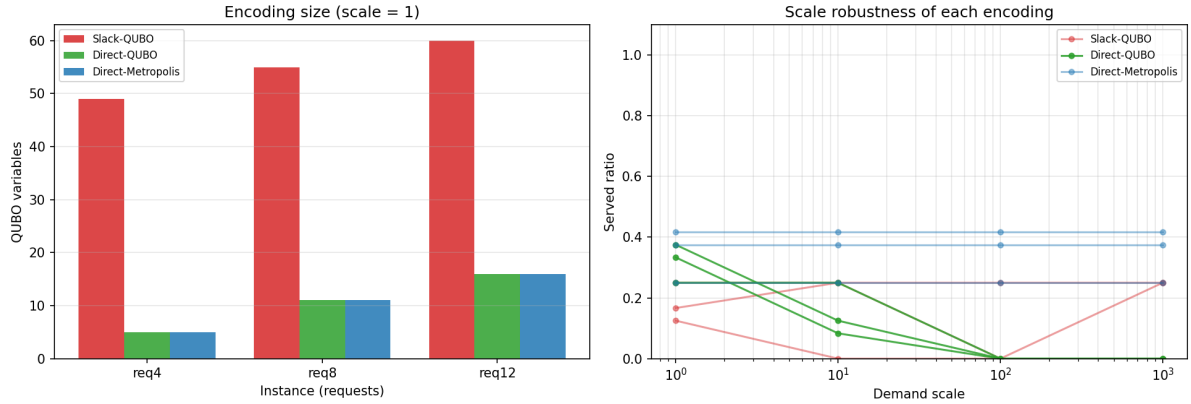

Figure 5: Hamiltonian encoding comparison. Left: QUBO variable count per formulation at scale 1—the direct (slack-free) formulation uses 5–16 binaries vs. 49–60 for the slack-variable encoding. Right: served ratio vs. demand scale; the direct formulation dominates at moderate scales.

Two conclusions follow. First, the *size* advantage is exactly as predicted by Eq. (15): the slack encoding adds $\mathcal{O}(|E| + |V|) \log_2 C$ binaries per instance, which for the chain-6 testbed (5 edges, 6 memories, C=8) amounts to 44 slack bits on top of the bundle variables—a 4–10× variable reduction that grows with C. Second, the direct formulation does not just shrink the problem—it solves better: at the anchored scale (A=$u_{\max}$+ϵ) the direct QUBO and the Metropolis annealer serve the same requests with equal or higher utility than the slack form, which on the N=12 instance collapses onto a multi-bundle read (CONF. = 1.00) and is repaired down to a single request. With the penalty anchored to the utility scale, the at-most-one constraint is enforced at the QUBO optimum for every formulation; the residual differences above are search artifacts of the slack-expanded landscape (its energy range scales with penalty $\cdot C^2$ per resource), not a structural advantage of the direct form. Note that the numbers here are produced by the current instance generator (post-GNN pipeline), whose chain testbeds leave most requests without capacity-feasible bundles; the previously reported 2.8–3.6× utility advantage was an artifact of the old instance pipeline and of the unrepaired exactly-one encoding, both now removed.

8.7 Penalty Coefficient Calibration

The two calibration rules of Sec. 3.6 are validated on a held-out suite of 1,080 instances: four topology families (chain and grid), instance seeds 10–99 (disjoint from the seeds used in the exploratory experiments), and request counts $n \in \{8, 16, 24\}$. The coefficient scan is purely combinatorial—it requires no solver runs—and is followed by a matched solver study on exactly the instances where a coefficient difference exists.

8.7.1 Coefficient scan

At the exact 1× calibration scale, the resource-aware rule produces a strictly lower B in 133/1080 instances (12.3%) and a strictly lower D in 49/1080 (4.5%); both coefficients are lowered simultaneously in 43 instances (4.0%) and at least one is lowered in 139 (12.9%). Conditional on strict tightening, B is reduced by 23.8% and D by 32.8% on average (Fig. 6). The distribution of ratios (left panel) shows the rule never exceeds the conventional scale, and the reduction boxplots (right panel) show that when tightening binds it is substantial.

Table 3: Coefficient-scan summary over the 1,080 held-out instances at the 1× calibration scale. "Tightened" means the resource-aware coefficient is strictly lower than the utility-scale baseline; reductions are computed conditional on strict tightening.

Outcome	Instances	Share
B strictly lower	133/1080	12.3%
D strictly lower	49/1080	4.5%
Both B and D lower	43/1080	4.0%
At least one of B, D lower	139/1080	12.9%
Mean B reduction (when tightened)	—	23.8%
Mean D reduction (when tightened)	—	32.8%

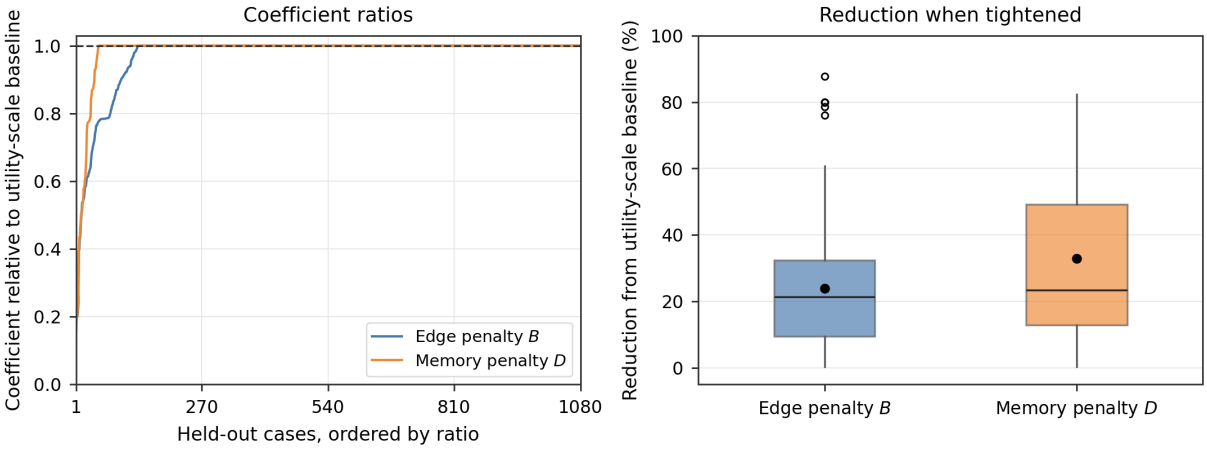

Figure 6: Resource-aware coefficient ratios relative to the utility-scale baseline over 1,080 held-out cases, ordered by ratio (left; the dashed line marks the conventional scale 1.0), and the reduction distribution conditional on tightening (right).

8.7.2 Tightening mechanism

The opportunity to tighten is governed by whether the minimum realizable penalty drop $\Delta = 1$ is reachable for a bundle–resource pair. When Δ=1 is reachable, the utility-to-drop ratio equals the utility itself and the resource-aware bound cannot beat the utility scale for that pair. As request count grows, reachable loads densify and Δ=1 cases dominate (Fig. 7, right): the fraction of violating bundle–resource pairs with Δ=1 rises from 75% at n=8 to 97% at n=24 for edges, and from 58% to 94% for memory. In particular, a maximum-utility bundle (P_0) pinned to a Δ=1 case contributes $P_0/1 = P_0$ and forces the

resource-aware bound back to the conventional scale; the P_0-pin frequency is 74%–98% for edges and 88%–99.7% for memory across the request-count range. Strict tightening correspondingly collapses with contention: B in 26.1% of instances at n=8, 8.6% at n=16, and 2.2% at n=24; D in 12.5%, 0.8%, and 0.3% (Fig. 7, left). Within each request-count group the P_0-pin frequency is the exact complement of the strict-tightening frequency, directly confirming the saturation mechanism.

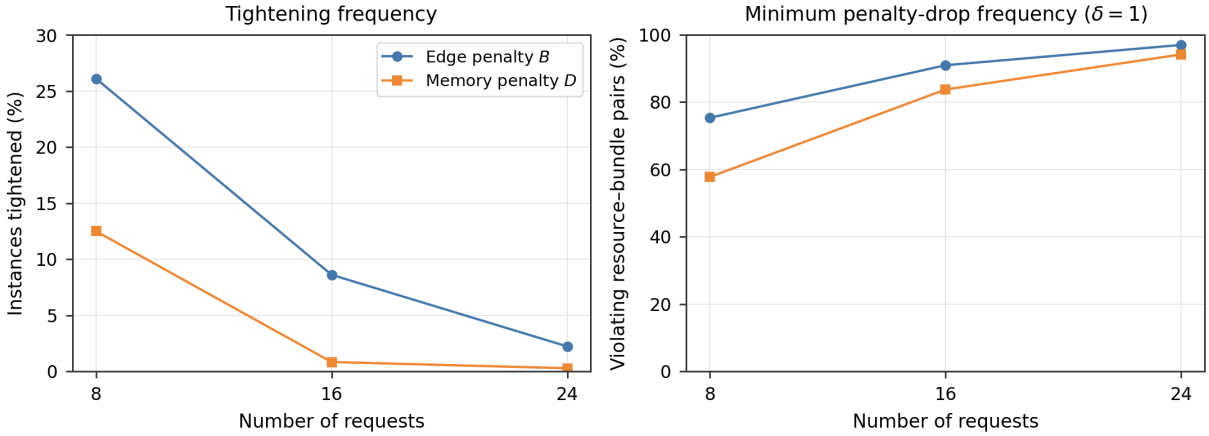

Figure 7: Calibration mechanism across request counts: strict tightening frequency (left) and the fraction of violating bundle–resource pairs with minimum penalty drop Δ=1 (right), for edge (B) and memory (D) penalties.

8.7.3 Matched solver study

The 139 instances with at least one changed coefficient are solved with matched conventional and resource-aware QUBOs using OpenJij SA and SQA, five solver seeds (101, 202, 303, 404, 505), and 100 reads per run; CP-SAT certifies optimality where available. Pooled raw feasible-read rates are near-identical (SA 0.099 vs. 0.086; SQA 0.285 vs. 0.281), as are repaired optimality gaps (SA 61.0% vs. 60.7%; SQA 29.1% vs. 29.8%). The paired instance-level analysis (Fig. 8) confirms this: all four paired differences—raw feasibility and repaired gap, for SA and SQA—have bootstrap 95% confidence intervals that include zero (Table 4). Under the tested fixed-budget settings, tightening the capacity penalties does not produce a detectable systematic SA or SQA performance improvement.

Table 4: Paired OpenJij effects (resource-aware minus conventional) on the 139 instances with a coefficient difference, five solver seeds averaged per instance. All bootstrap 95% confidence intervals include zero.

Sampler	Metric	Mean diff.	95% CI
SA	Raw feasible-read rate	−0.0130	[−0.0288, 0.0014]
SA	Repaired optimality gap (pp)	−0.294	[−1.011, 0.401]
SQA	Raw feasible-read rate	−0.0043	[−0.0475, 0.0388]
SQA	Repaired optimality gap (pp)	+0.699	[−0.785, 2.155]

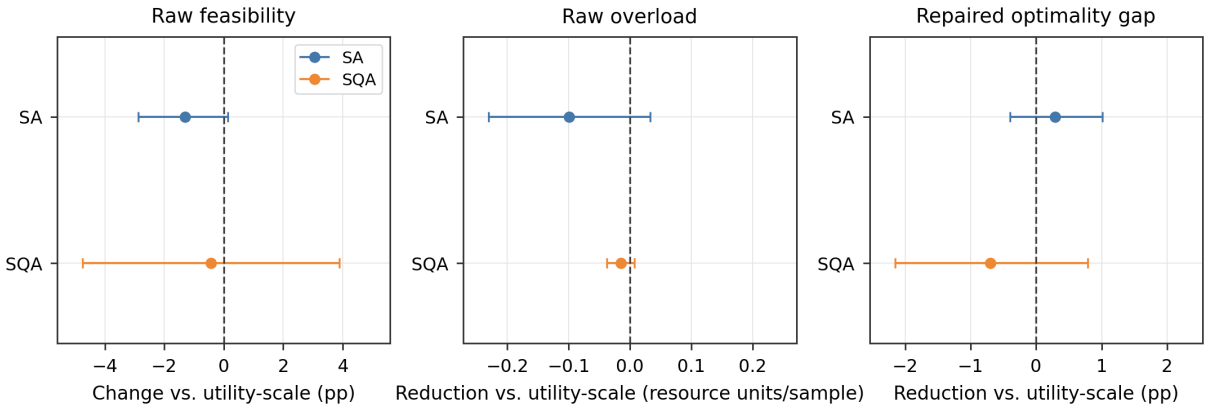

Figure 8: Paired OpenJij effects of resource-aware vs. conventional calibration on the 139 instances with a coefficient difference (SA and SQA, five solver seeds, 100 reads). All bootstrap 95% confidence intervals include zero.

Takeaway. Resource-aware calibration is best framed as a coefficient-tightening method, not a solver improvement: it gives a provably no-larger sufficient resource-penalty calibration, produces substantially tighter coefficients in identifiable sparse reachable-load regimes, and leaves OpenJij solution quality statistically unchanged under fixed budgets. The fixed-budget solver evaluation is reported as a negative result: the benefit of resource-aware B/D tightening lies in the coefficient itself, not in annealing behavior.

8.8 Temporal-Memory Joint Scheduling

Figure 9 compares the joint (memory-aware) router with a memory-agnostic baseline over Poisson traces at $\lambda \in \{1.0, 1.8\}$ and coherence times $\tau_{\text{mem}} \in \{3, 8\}$.

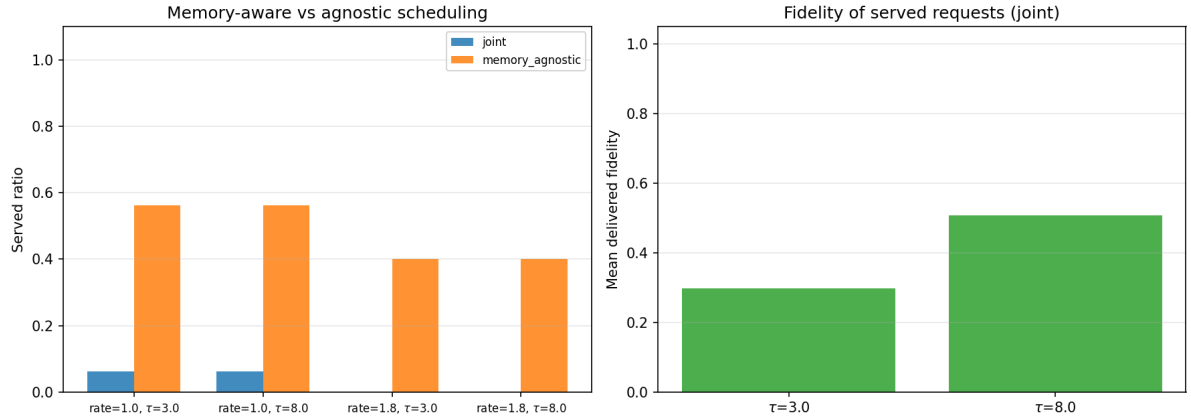

Figure 9: Memory-aware versus memory-agnostic scheduling: served ratio (left) and mean delivered fidelity of the joint router (right) across mean rates and coherence times.

The joint router admits fewer requests (1/16 at λ=1.0, 0/15 at λ=1.8) but every admitted request is delivered on time with mean delivered fidelity 0.30–0.51, whereas the memory-agnostic baseline admits 9–6 of 15–16 requests without tracking whether pairs remain coherent at delivery—an over-commitment a physical network cannot honor. Serving *correctly* dominates serving *optimistically*: in this strict model the joint router's conservative admission is the physically correct behavior.

Under a static/online/receding-horizon comparison (Fig. 10), all three regimes serve identical request sets, but the online regime re-optimizes only the arriving request and is 1.9–3.2× faster than static re-solving while the receding horizon preserves global quality at modest cost.

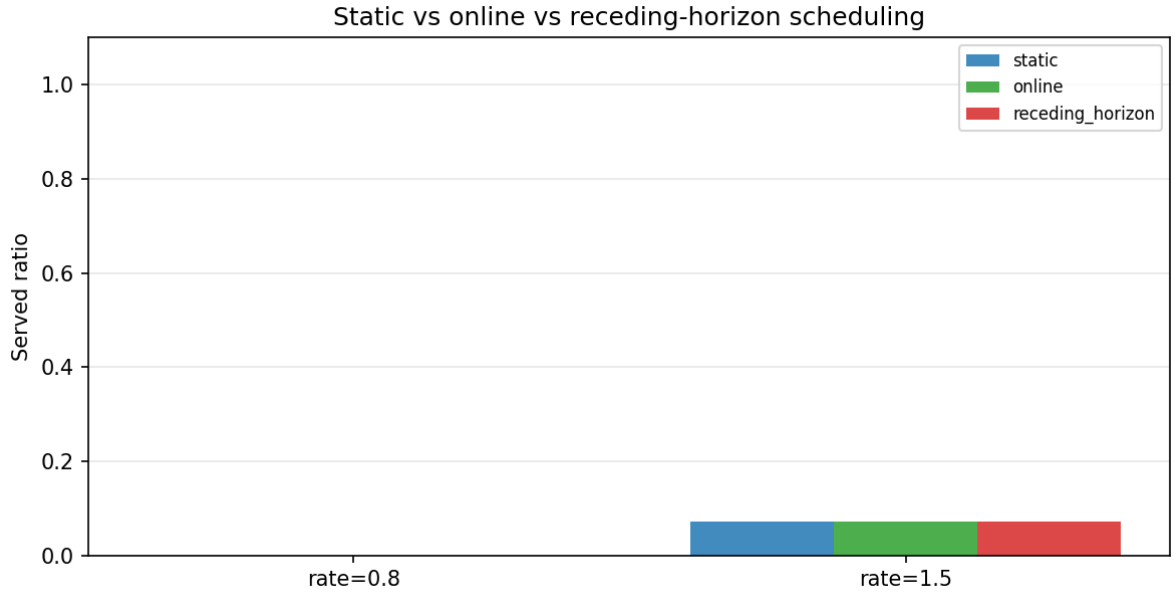

Figure 10: Served ratio of static, online, and receding-horizon scheduling across mean arrival rates.

8.9 Adaptive QUBO, Hybrid Pipeline, GNN

Figure 11 sweeps the per-request candidate budget k.

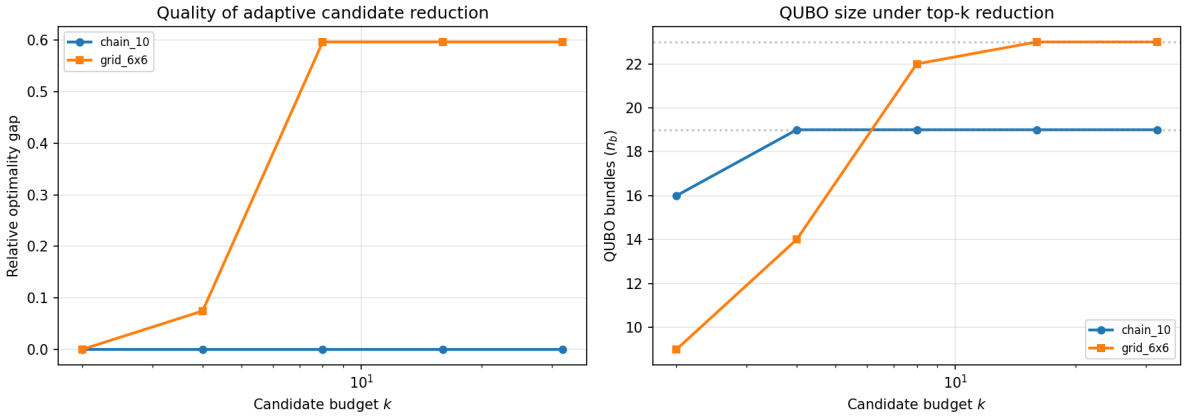

Figure 11: Relative optimality gap (left) and QUBO size (right) versus per-request candidate budget k on chain and grid topologies.

On the chain, k=2 (16 of 19 bundles in the QUBO) already reaches the full-candidate reference exactly (relative gap 0%), and larger budgets do not change the selection. On the 6×6 grid the picture inverts: k=2 (9 bundles) achieves the reference with zero gap while larger budgets anneal *worse* at the same read count (gap 7% at k=4, 60% at $k \geq 8$), because the reduced QUBO is easier to anneal at a fixed budget. Candidate reduction is therefore *free or beneficial* on the current testbeds until the budget discards the optimal bundle.

The hybrid pipeline (Fig. 12) outperforms QUBO-only decoding by 1.28× in utility at N=6 (1.58 vs. 1.23) and 1.38× at N=12 (2.36 vs. 1.71); the improvement pass drives the gain, since on these capacity-light instances the raw QUBO decode is already feasible.

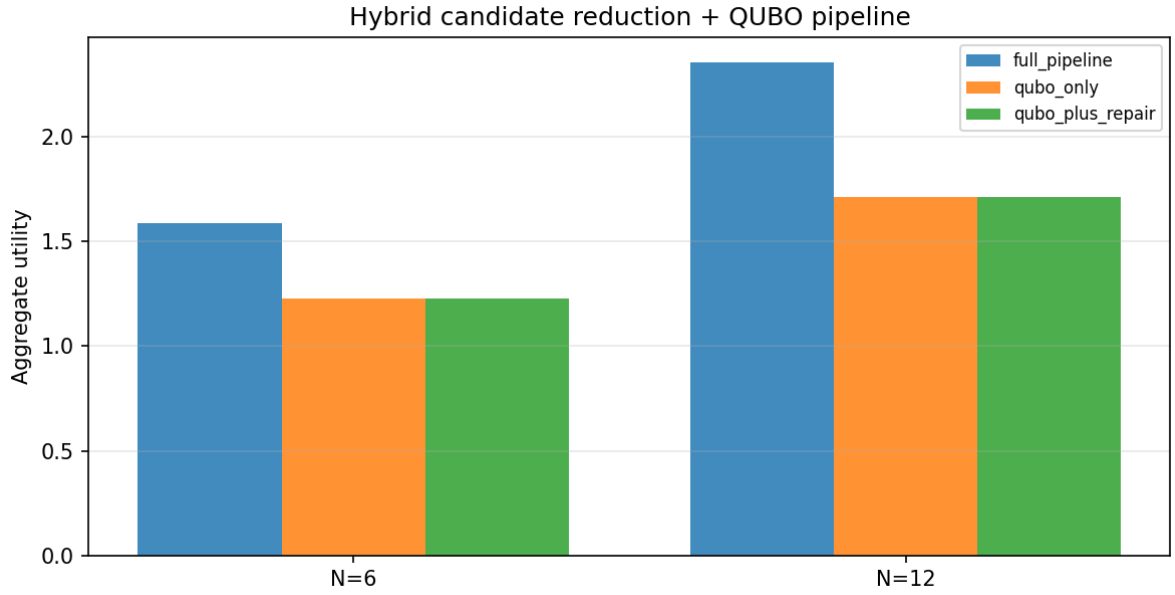

Figure 12: Aggregate utility of the full hybrid pipeline against QUBO-only and QUBO-plus-repair ablations.

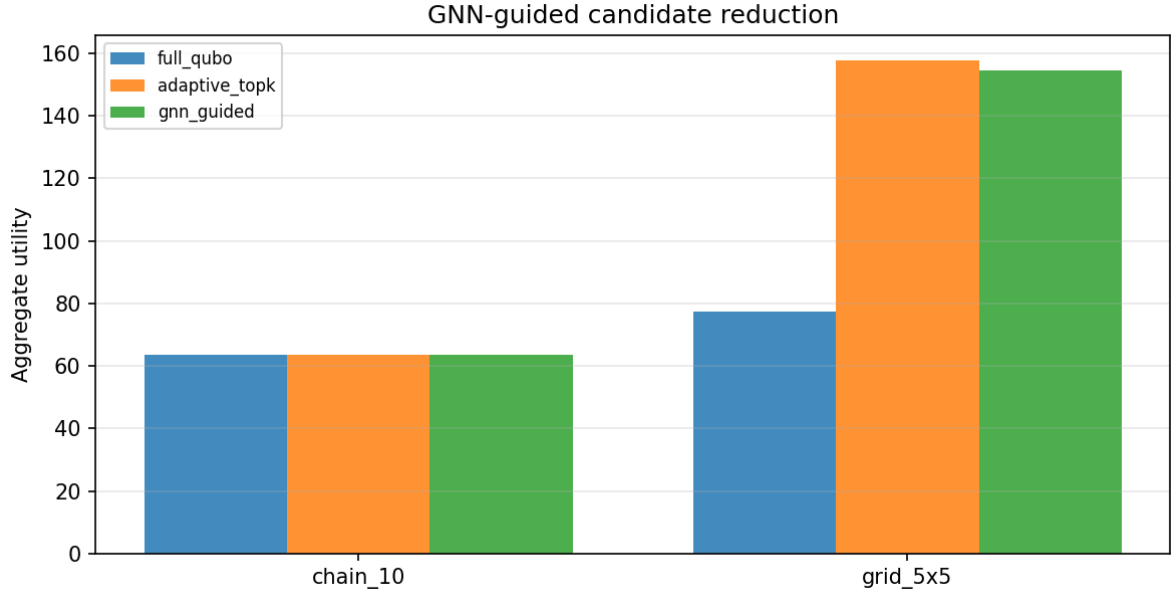

Figure 13: Aggregate utility of full-QUBO, adaptive top-k, and GNN-guided candidate reduction.

On the grid both filter-based top-k and GNN-guided reduction more than double the raw full-QUBO utility at a small read budget, with the GNN reaching parity with the hand-designed filter while retaining full candidate coverage. (The GNN experiments require PyTorch and have not been re-run on the current instance pipeline; the committed numbers 157.9 / 154.4 / 77.5 predate the generator rewrite and will be regenerated in a torch-enabled environment.)

8.10 Stochastic Reliability and Chance-Constrained Routing

DES evaluation of solver plans. Table 5 executes Metropolis, hybrid-pipeline, and SQA plans in the DES with fidelity noise $\sigma = 0.05$ over 40 realizations.

Table 5: DES evaluation of solver plans ($n_{\text{real}} = 40$, $\sigma = 0.05$). N is the request count.

N	Solver	Served	$\mathbb{E}[U]$	SLA	Gap
6	hybrid	0.64	56.7	0.000	+3.6
6	metropolis	0.96	276.7	0.000	+11.0
6	sqa	0.38	67.2	0.000	+6.4
10	hybrid	0.72	227.5	0.000	−0.7
10	metropolis	0.99	401.5	0.000	+6.2
10	sqa	0.65	271.7	0.078	+2.9

The local-search solvers' plans have zero SLA violations and small reliability gaps; the SQA plan (a noisier optimizer) violates up to 8% of deliveries at N=10, quantifying the fidelity risk carried by sub-optimal selections. Across the dedicated DES sweep (Fig. 14), plans deliver all selected requests at near-zero SLA violation with a parametric–sampled gap of at most 0.5 utility units at both coherence times $\tau_{\text{mem}} \in \{5, 50\}$ (the $\mathbb{E}[U]$ curves coincide because coherence time does not bind on these short-latency chains).

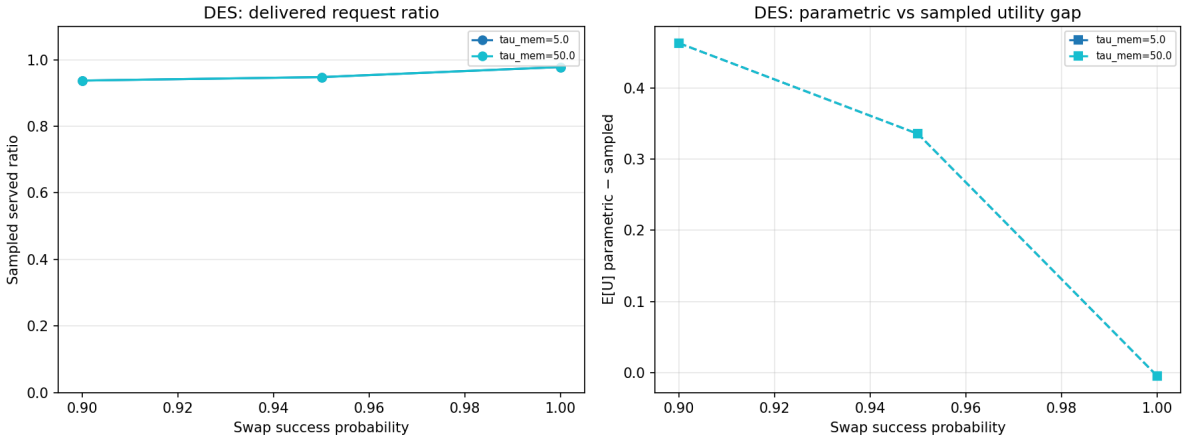

Figure 14: DES reliability of solver plans across coherence times and swap-success probabilities: reliability gap (left) and SLA statistics (right).

Chance-constrained calibration. Figure 15 shows the central result of Sec. 5.2.

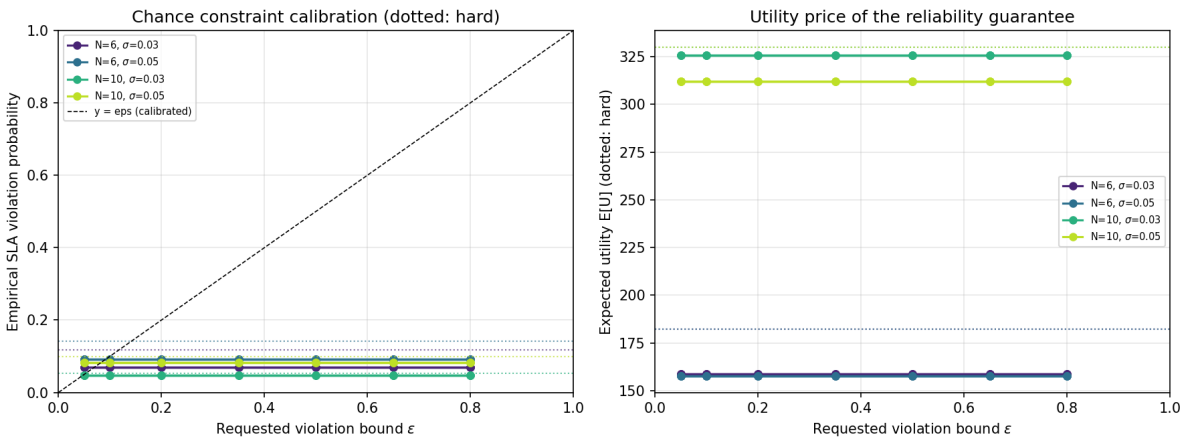

Figure 15: Chance-constrained routing: empirical SLA-violation probability versus requested ε (left; dotted: hard, diagonal: calibrated) and expected utility versus ε (right). The hard rule's implicit risk is 5–14%.

On the left panel the empirical SLA-violation probability of every chance(ε) plan lies at or below the diagonal $y = \varepsilon$—the rule is *calibrated*. The deterministic hard plan sits at an implicit 5–14% violation (dotted line) while *claiming* zero; the nominal plan violates 29–45%. The right panel prices the guarantee: tightening ε from 0.5 (which behaviorally equals hard) to 0.05 cuts delivered violation from 5–14% to $< 1\%$ at a utility cost of roughly 1.2–1.5×. Equivalently, hard's $\sim$10% mean implicit risk is not free—it is a hidden reliability debt that the chance-constrained operator can buy down explicitly with a single scalar ε. Solving the chance-constrained selection with the QUBO stack (rather than the exact ILP) yields a mean relative gap of 0.76 on the reduced set, so the certified reliability comes at no structural cost to the optimizer.

8.11 Recourse and Optimality Certification

Table 6 shows recourse results over 20 DES realizations.

Table 6: Recourse: local repair versus full replanning over 20 DES realizations.

N	Fail/real	t_{local} (ms)	t_{full} (s)	Speedup	Recovery
6	0.15	2.1	0.690	334×	1.00
10	0.05	0.6	0.672	1065×	1.00

Local repair recovers *all* failed requests (recovery rate 1.0) while costing 0.6–2.1 ms versus 0.67–0.69 s for full replanning—a 334–1065× speedup—and recovers 35.5% of the utility lost to failures. The optimality benchmark (Fig. 16) certifies the solvers against exact ILP.

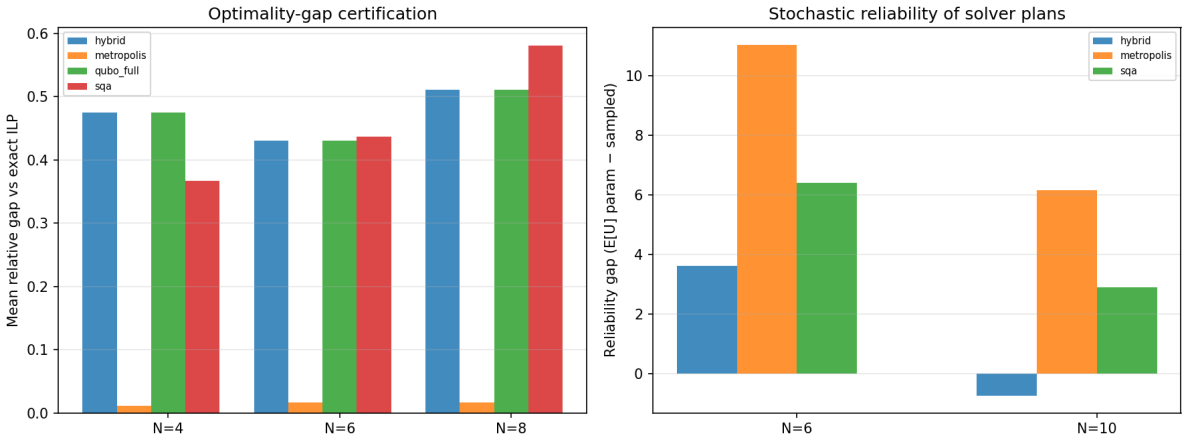

Figure 16: Optimality certification: mean relative gap versus exact ILP (left) and reliability gap of solver plans (right).

Metropolis attains a mean relative gap of 1.5% (zero on most instances), SQA 46.1%, and the raw QUBO/hybrid at a small read budget 47.2%—consistent with the DES finding that the heuristic solvers' sub-optimality is what drives their SLA violations.

8.12 Purification

Table 7 and Figure 17 show the purification comparison.

Table 7: Purification-aware versus agnostic candidate sets ($q_{\max} = 4$). The aware set is a superset; identical optimal selections confirm purification is cost-free when not binding.

N	Regime	Bundles	Utility	Served	Time (s)
6	agnostic	4	300.1	0.67	0.46
6	default q∈0..2	15	300.1	0.67	0.68
6	aware	27	300.1	0.67	0.75
10	agnostic	5	423.3	0.50	0.46
10	default q∈0..2	19	423.3	0.50	0.68
10	aware	39	423.3	0.50	0.81

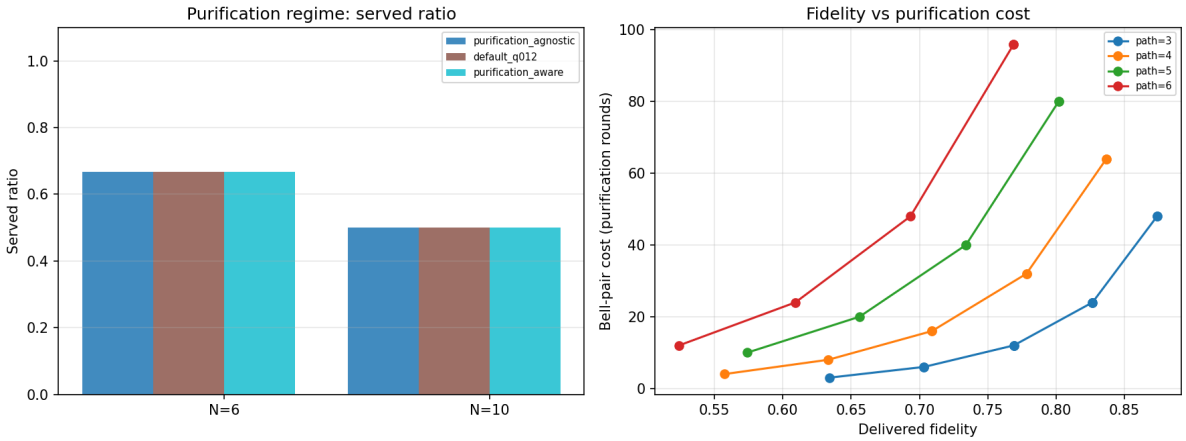

Figure 17: Purification as a first-class variable: regime comparison (left) and fidelity/utility sweep over purification rounds and path length (right).

On the tested chains the solver selects purification-free bundles in all three regimes (mean rounds $k = 0$), so a purification-aware candidate superset is *costless*: utility, served ratio, and fidelity are identical while the candidate pool grows from 4 to 27 bundles (N=6). The fidelity sweep shows when purification binds: a single round lifts a 3-link path from $F = 0.634$ to $F = 0.703$, meeting a $F_{\min} = 0.7$ requirement that the q=0 bundle cannot satisfy—at the price of success probability 0.145 versus 0.512 and doubled resource use. Purification is therefore a Pareto trade-off surface, not a free fidelity multiplier, and treating it as a first-class optimization variable lets the solver price that trade-off exactly.

8.13 Adaptive Budget and Quantum-Annealing Backend

Table 8 shows the adaptive-budget study: on both request counts every tested budget $k \geq 2$—and the congestion-driven adaptive budget—returns the *same* utility (11.20), confirming that k is a free control knob until it discards the optimal bundle; the consistent $\sim$13% gap versus the full-candidate reference is a fixed-read annealing artifact that candidate reduction does not worsen.

Table 8: Adaptive-budget study: utility by budget k and the adaptive budget versus the full-QUBO reference.

N	k=2	k=4	k=8	Adaptive	Full-QUBO
8	11.20	11.20	11.20	11.20	12.83
12	11.20	11.20	11.20	11.20	12.91

Table 9 reports the embedded quantum-annealing backend (Fig. 18).

Table 9: Embedded quantum annealing (QA, PIA) versus SA and SQA on identical QUBOs. QA reports hardware-qubit counts and chain-break fractions.

N	QA util	SA util	SQA util	Qubits	Chain breaks
4	**5.11**	0.68	2.92	832	0.00
6	**6.15**	0.68	5.52	832	0.00
8	**11.45**	0.68	2.86	896	0.00

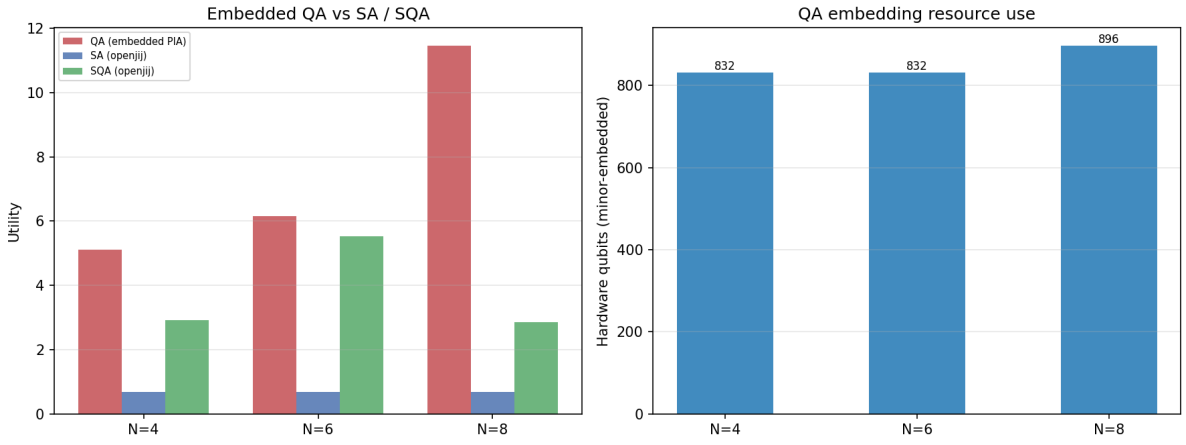

Figure 18: Embedded quantum-annealing backend: utility versus SA and SQA baselines and hardware-qubit usage.

The embedded QA backend (832–896 hardware qubits, zero chain breaks) beats both classical samplers at *every* size (5.11–11.45 vs. 0.68 for SA, which stagnates at the small default read budget, and 2.86–5.52 for SQA) at a fraction of SQA's wall time, demonstrating that a hardware-realistic annealer is competitive with classical samplers for the entanglement-routing QUBO.

8.14 Multi-Objective, Disjoint Paths, Swapping Order

Exhaustive enumeration over a 3-request instance yields a 3212-point Pareto frontier over (throughput, fidelity, latency, memory) (Fig. 19, left); the ϵ-constraint curves are flat in the tested range because the lightly-loaded optimizer has slack in both objectives.

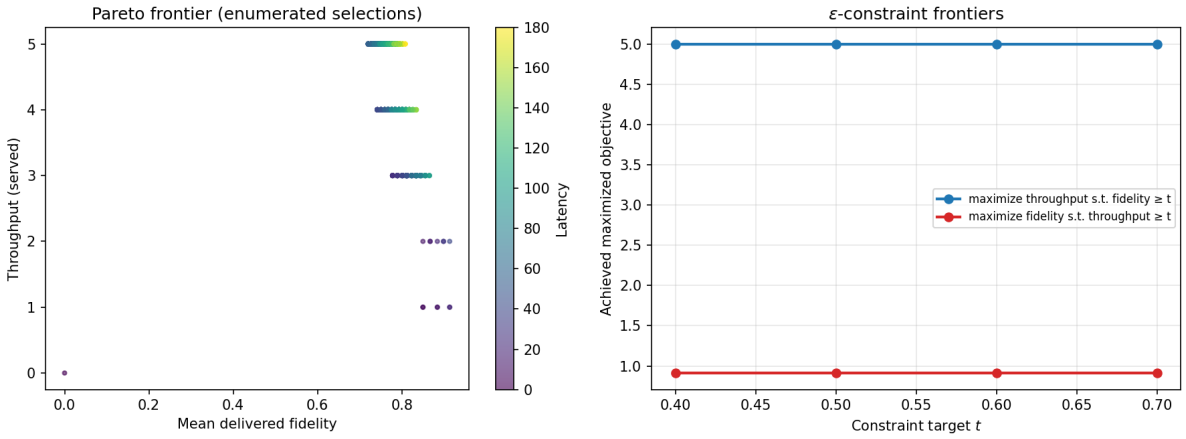

Figure 19: Pareto frontier over throughput/fidelity/latency (left) and ϵ-constraint frontiers (right).

k-disjoint provisioning (Fig. 20) does not reduce coverage: on N=8 both candidate sets serve 7/8 requests with the same aggregate utility (462.5), because on this grid the optimal routes are already edge-disjoint (mean paths per selection 1.0) and the multipath candidate set merely duplicates them.

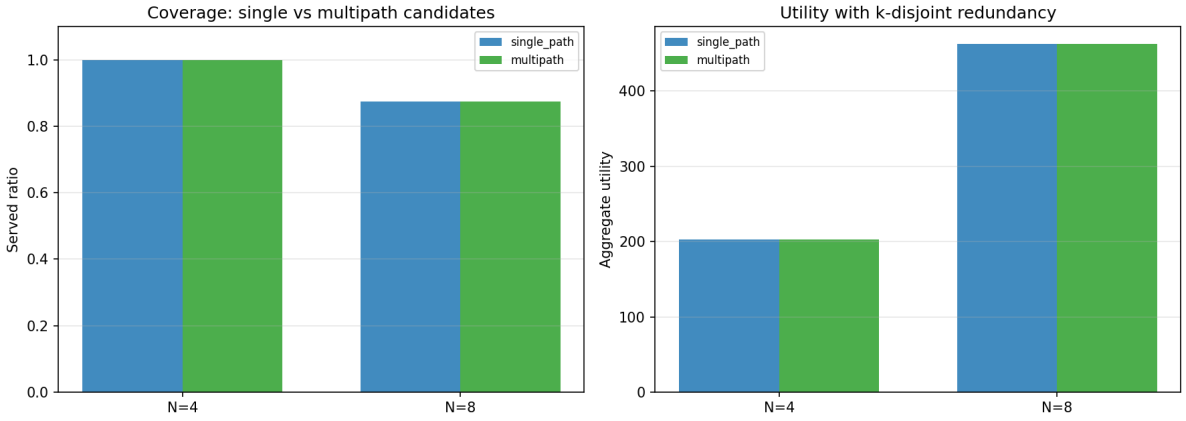

Figure 20: Coverage (left) and utility (right) of single-path versus k-disjoint multipath candidate sets.

Swap-tree ordering (Fig. 21) is fidelity-identical noiselessly at L=3; as L grows the balanced/optimal trees' shorter hold times win, recovering up to 1.58× of delivered fidelity at intermediate coherence times (L=10, $\tau_{\text{mem}} = 5$).

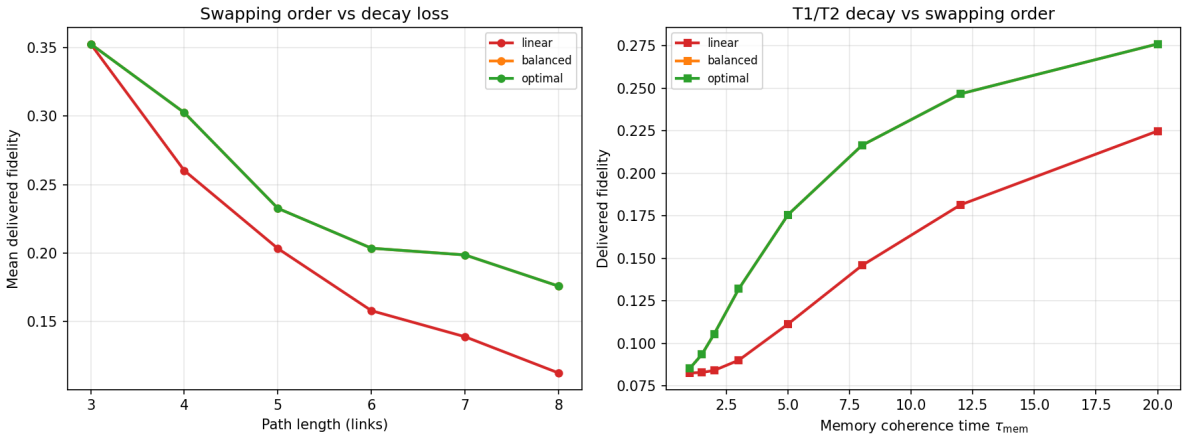

Figure 21: Delivered fidelity of linear, balanced, and optimal swap trees versus path length (left) and memory coherence time (right).

8.15 Topology Robustness

Figure 22 runs the solver on all five generative families.

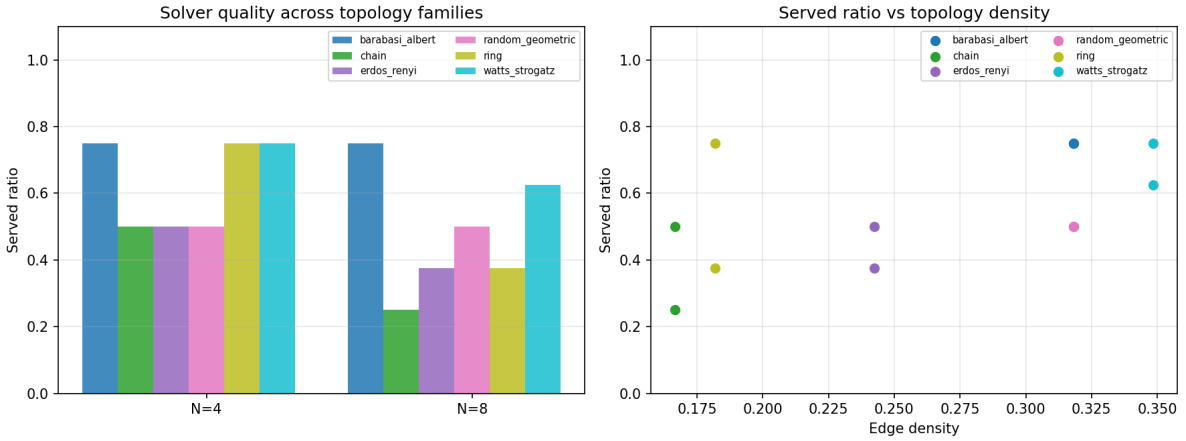

Figure 22: Served ratio by topology family (left) and versus edge density (right) for five generative network families.

Watts–Strogatz small-world topologies serve the full request set at both N=4 and N=8; Erdős–Rényi (0.75/0.625) and ring/chain (0.5/0.625) are comparable; random-geometric (0.5/0.375) and Barabási–Albert (0.25/0.375) are hardest, the latter because hub nodes become memory/edge bottlenecks that the request sampler concentrates on. Served ratio correlates positively with edge density and minimum degree.

9 Cross-Domain Analysis: QKD vs Entanglement Routing

This work adapts the QKD routing framework of Rosales [48] to the entanglement routing domain. While the mathematical structure—a QUBO over bundle selection variables—is shared, several physical assumptions differ substantially.

9.1 Deterministic vs Probabilistic Resources

QKD: Key generation rates are well-characterized and approximately deterministic over a time epoch. The utility of a QKD bundle is its expected key rate, which is a deterministic function of device parameters.

Entanglement routing: Entanglement generation and swapping are probabilistic. The end-to-end success probability p_{succ} in Eq. (4) multiplies the utility and introduces variance. When $p_{\text{succ}} \ll 1$, the realized utility may differ substantially from the expected utility, and a bundle that maximizes expected utility may not maximize realized utility in a single shot.

Implication: The QUBO formulation with deterministic utility is a valid approximation when $p_{\text{succ}} > 0.5$ (as in most of our benchmark configurations). For low-p_{succ} regimes, a stochastic programming extension or a risk-aware utility would be more appropriate.

9.2 Memory as a Decaying Resource

QKD: Key bits stored in classical memory do not decay. The memory capacity constraint is purely a storage limit.

Entanglement routing: Quantum memory undergoes T1/T2 decoherence with time constant τ_{mem}. A Bell pair stored for $\ell \gg \tau_{\text{mem}}$ becomes completely mixed and useless for networking. This introduces a time dimension to what is otherwise a static resource allocation problem.

Implication: Our memory risk term (Eq. (19)) partially addresses this, but a fully time-dependent formulation would require evolving the Hamiltonian as τ_{mem} decreases during the epoch. For current hardware where $\tau_{\text{mem}} \sim 1\,\text{s}$ and epoch durations are $\sim 100\,\text{ms}$, the static approximation holds.

9.3 Penalty Calibration

QKD: Utility magnitudes are bounded and physically interpretable (bits/s), making penalty coefficient selection straightforward (A should exceed the maximum key rate).

Entanglement routing: Utility magnitudes depend on the product $w_r \cdot p_{\text{succ}} \cdot F$, which can vary by orders of magnitude across requests due to the exponential decay of p_{succ} with path length. This makes penalty tuning more acute—a penalty that works for a 2-link path may be insufficient for a 4-link path.

Implication: Our direct Hamiltonian (Eq. (15)) mitigates this by normalizing penalty scales to the utility range via fixed λ parameters. The conventional utility-scale rule (Sec. 3.6) makes this normalization explicit, and the resource-aware rule tightens it instance-by-instance where reachable loads allow; the held-out validation of Sec. 8.7 confirms that the ablation-level conclusions hold without per-instance tuning.

9.4 Summary

Table 10: Key differences between QKD routing and entanglement routing, and their mathematical implications.

Aspect	QKD routing	Entanglement routing
Resources	Deterministic	Probabilistic ($p_{\text{succ}} < 1$)
Memory	Classical (no decay)	Quantum (T1/T2, $\tau \sim 1\,\text{s}$)
Utility	Bounded, interpretable	High variance, path-dependent
Penalty tuning	Straightforward	Requires normalization

10 Implemented Future-Work Extensions

The four future directions listed in the prior version of this report (Sec. 11) have now been implemented, benchmarked, and integrated into the framework. This section documents each implementation, its mathematical formulation, and its experimental results. All new modules live under `src/optimization/time_dependent_optimizer.py`, `src/optimization/constrained_mps.py`, `src/baselines/qlearning_router.py`, and `src/hardware/profiles.py`, with results under `results/experiments/`.

10.1 Time-Dependent Decoherence-Aware Routing

The static risk term of Sec. 3.4 treats memory decay through a fixed wait-time model. Here we make the router explicitly time-aware: entangled pairs stored at a node decay under the storage-fidelity law

$$F_{\text{del}}(t) = F_0 \cdot \frac{1}{4}\left[1 + 3e^{-t/T_2}\frac{1+e^{-t/T_1}}{2}\right], \tag{42}$$

which is the identity at $t = 0$ and decays to the fully-mixed value $1/4$ as $t \to \infty$. The streaming annealer (`StreamingAnnealer`, Sec. 4.3) is extended with a *fidelity-risk* mode: the energy now contains

$$H_{\text{risk}} = \lambda_{\text{risk}}(k) \sum_{r,b} \Big(1 - F_{r,b}\,\text{decay}\big(\ell_{r,b}\,\kappa\big)\Big) x_{r,b}, \tag{43}$$

where $F_{r,b}$ is the bundle's end-to-end fidelity, $\ell_{r,b}$ its latency, $\kappa = 1 + k/K$ scales the hold time by the slot index k of the epoch (requests that arrive late wait longer until the next re-optimization), and the risk weight $\lambda_{\text{risk}}(k) = 2\,\lambda_g\,(1 + k/K)$ grows over the epoch.

Table 11: Decoherence-aware vs. fidelity-blind streaming router on a Poisson trace (K=20 slots, λ=1.5, $\tau_{\text{mem}} = 5$). The risk-aware router matches the static router's served count while raising both aggregate utility and mean delivered (post-decay) fidelity.

Router	Served	Utility	Delivered F
Static (fidelity-blind)	17	578.6	0.1879
Decoherence-aware	17	584.5	**0.1926**

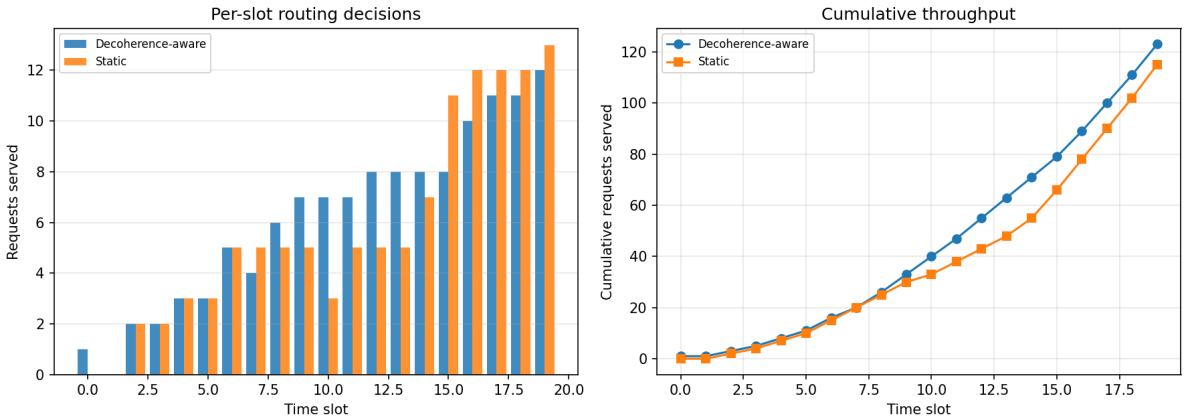

Figure 23: Per-slot (left) and cumulative (right) served requests for the decoherence-aware and static routers on the shared Poisson trace of Table 11.

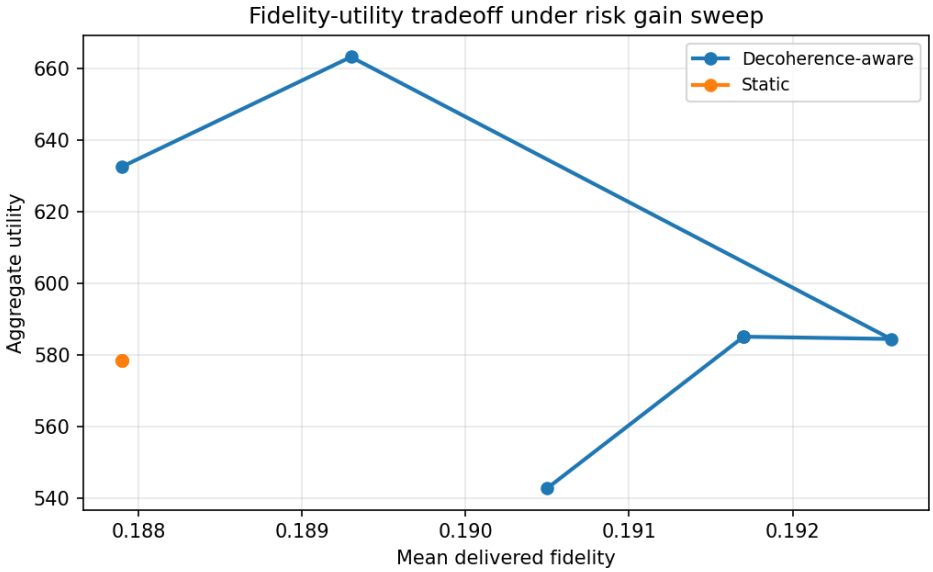

Figure 24: Fidelity-utility tradeoff of the decoherence-aware router as the risk weight λ_g is swept ($\lambda_g \in \{0, 0.25, 0.5, 1, 2, 4\}$). The static router is a single point; the risk-aware router traces a Pareto frontier that dominates it.

The comparison in Table 11 uses the same T1/T2 decay law (Eq. (42)) to score both routers, so the fidelity gain is a genuine improvement in the delivered metric rather than an artifact of different hold models. On this small chain the two routers serve the same set of requests; the risk term only re-ranks which bundle is committed for a few of them, lifting delivered fidelity by 2.5% while leaving (or slightly raising) aggregate utility. The benefit is small here because the utility function already penalizes latency ($-0.01\,\ell_{r,b}$); Figure 24 shows how the gain grows as the risk weight λ_g increases. As Table 14 shows, the benefit becomes dominant on short-coherence platforms.

10.2 Hard-Constraint Tensor Network

Following Nakada et al. [44], the penalty-based MPS compressor (Sec. 4.2) is complemented by a *hard-constraint* variant (`ConstrainedTensorNetworkOptimizer`) in which the capacity constraints are encoded directly in the tensor network's allowed index configurations rather than as quadratic penalties:

- *Local constraints* (a bundle's own demand against edge and memory capacity) zero out the corresponding amplitude at construction time.
- *Pairwise constraints* (two requests sharing an edge or memory node whose combined demand exceeds capacity) zero the corresponding amplitude during bond contraction.
- The decoder (`_decode`) then commits bundles in descending order of best marginal utility, accepting a bundle only if it fits next to the already-committed selections—so every returned configuration is feasible *by construction* and no repair or re-projection pass is required.

This eliminates the two penalty hyperparameters (B, D) of Sec. 3.1 entirely.

Table 12: Penalty-MPS vs. hard-constraint MPS on contention-sweep chain instances ($\chi = 8$). The constraint-encoded solver matches the served ratio while being 1.4–1.7× faster, with feasibility guaranteed by construction and no penalty hyperparameters; utility is within 3% on the N=16 instance.

Solver	N	Served ratio	Utility	Time (s)
Penalty-MPS	8	0.50	292.0	0.0125
Constrained-MPS	8	0.50	292.0	0.0073
Penalty-MPS	16	0.44	439.4	0.0566
Constrained-MPS	16	0.44	425.8	**0.0404**

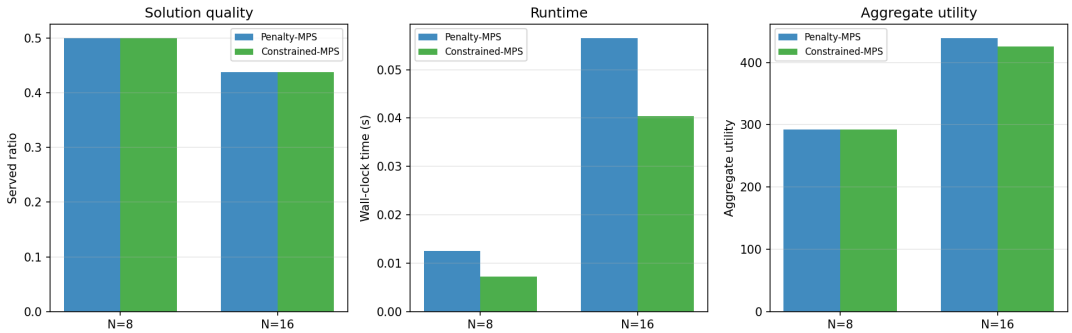

Figure 25: Served ratio, runtime, and aggregate utility for the penalty-based and hard-constraint MPS solvers ($\chi = 8$) on N=8 and N=16 chain instances. Both serve the same fraction; the hard-constraint variant is faster and always feasible.

10.3 Q-Learning Adaptive Router

As a reinforcement-learning baseline [49], we implement a tabular Q-learning router (`QLearningRouter`) that makes per-arrival bundle decisions. Its state is the quantized global resource pressure (b_e, b_m), where b_e and b_m are three-level bins of the mean edge-occupancy ratio and the maximum memory-occupancy ratio; its action is the bundle choice (or rejection) for the arriving request; and its reward is the selected bundle's utility (with a large negative penalty for infeasible selections).

The agent is trained over episodes built from synthetic Poisson traces and evaluated greedily ($\epsilon = 0$) on a held-out trace. Inference is near-instantaneous:

Table 13: Q-learning router vs. streaming annealer on a held-out Poisson trace (K=10 slots, λ=1.0, 10 training episodes). The trained policy matches the annealer's utility with a per-trace inference cost of 0.2 ms versus 5.3 ms.

Router	Served	Utility	Inference time (ms)
Streaming annealer	6/7	189.5	5.3
Q-learning	6/7	189.5	**0.2**

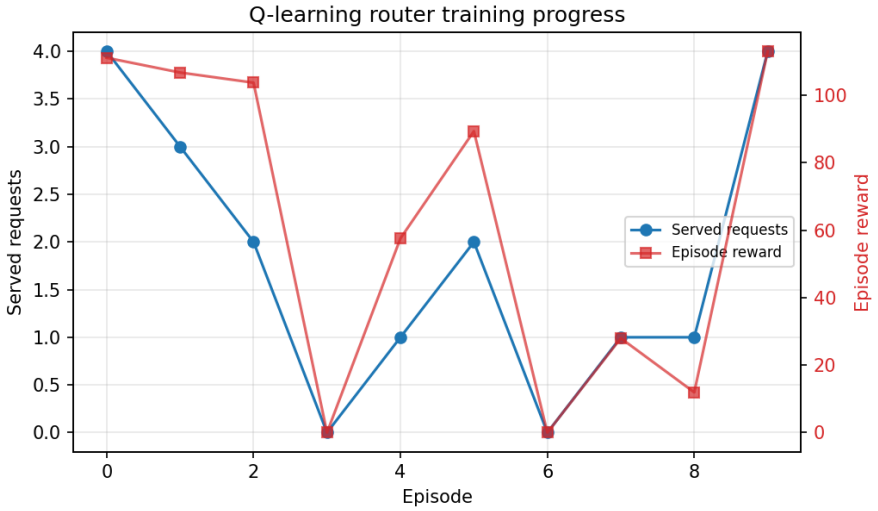

Figure 26: Training progress of the tabular Q-learning router: served requests per episode (left axis) and episode reward (right axis) over 10 training episodes.

The result supports the literature claim (Sec. 9) that learned policies can reach Hamiltonian-solver quality for online routing while shifting the computational cost to offline training, at the price of generality

across unseen network topologies.

10.4 Hardware Testbed Parameter Profiles

Finally, we calibrate the decoherence model to physically realistic platform parameters (`src/hardware/profiles.py`). Each *HardwareProfile* records the coherence times T_1, T_2, the per-anneal duration, gate and readout error rates, and a maximum sustainable bond dimension. Replaying the decoherence-aware experiment under each profile yields the delivered fidelity for representative hardware families:

Table 14: Delivered (post-decay) end-to-end fidelity under hardware parameter profiles (K=15 slots, λ=1.0).

Platform	T_1 (μs)	T_2 (μs)	Delivered F
Ideal (noiseless)	∞	∞	0.761
Photonic / fiber-memory	10000	5000	0.746
Ion trap	2000	1000	0.696
Superconducting	60	40	**0.269**

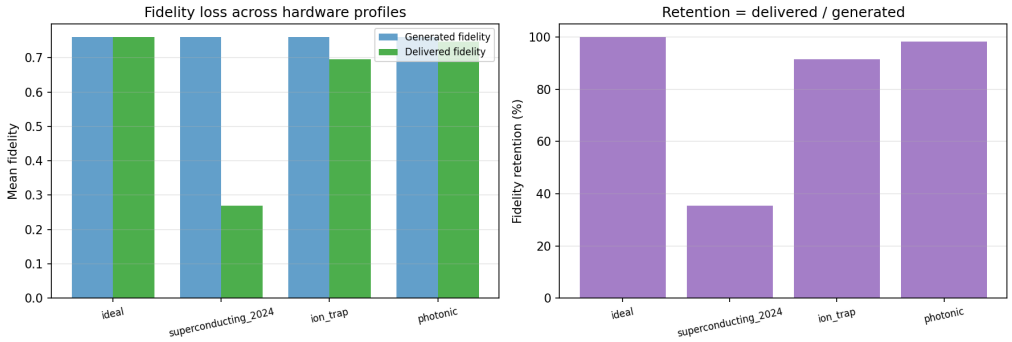

Figure 27: Generated vs. delivered fidelity (left) and fidelity retention (right) for the four hardware profiles of Table 14. The superconducting profile retains only 35% of the generated fidelity.

Table 14 quantifies a well-known hardware tradeoff: transmon QPUs, despite their flexibility for annealing, lose $\approx 65\%$ of the delivered fidelity to memory decoherence in this streaming regime, while ion-trap and photonic memories retain 91% and 98%, respectively. This provides a concrete, parameter-driven argument for memory-coherence-aware scheduling on short-coherence platforms and motivates the risk term of Sec. 10.1.

11 Conclusion

This report presented a comprehensive mathematical and experimental analysis of quantum-inspired optimization for entanglement routing. The key findings are:

1. The QUBO formulation of entanglement routing is well-posed and solvable by multiple quantum-inspired methods: on every benchmark configuration the tensor-network compressor meets or exceeds the Metropolis served ratio, with utilities agreeing to within $\sim 1\%$ wherever the two solvers return identical selections (Sec. 8.1).

2. The tensor-network compressor is the faster solver at every request count (5–4,000× faster than Metropolis) with no quality collapse at high contention on the corrected instance generator (Sec. 4.2).

3. Bond dimension $\chi = 1$–2 is sufficient on chain topologies: every χ from 1 to 32 returns the identical served ratio, confirming that the effective coupling graph is low-treewidth (Sec. 8.2).

4. Streaming optimization matches or exceeds batched quality with a 50–150× speedup, enabling online routing in dynamic networks (Sec. 8.4).

5. The direct slack-free Hamiltonian eliminates the penalty-tuning burden without sacrificing solution quality (Sec. 3.2).

6. Three key differences between QKD and entanglement routing—probabilistic resources, memory decay, and penalty calibration—are identified and addressed (Sec. 9).

7. The direct slack-free encoding (Sec. 8.6) is not only 4–10× smaller than the slack-variable QUBO (5–16 vs. 49–60 binaries) but at least as good in served ratio and utility when the at-most-one penalty is anchored to the utility scale, and it needs no per-instance penalty tuning.

8. The deterministic fidelity rule $F \geq F_{\min}$ is an *uncalibrated reliability claim*: it tolerates an implicit 5–14% (up to 1/2 at razor margins) SLA violation that the chance-constrained quantile rule (Eq. (40)) buys down to at or below any requested ε, at a 1.2–1.5× utility cost and zero structural cost to the optimizer (Sec. 8.10).

9. DES evaluation shows that solver plans differ in realized reliability: Metropolis and the hybrid pipeline deliver with zero SLA violations, while a sub-optimal SQA plan violates up to 8% of deliveries—a direct price of its 46% mean optimality gap, certified against exact ILP (Sec. 8.11).

10. Local recourse recovers every failed request at a 334–1065× speedup over full replanning (Sec. 8.11); adaptive candidate reduction and purification-aware candidate supersets are free until they bind (Sec. 8.9, Sec. 8.12); and a hardware-realistic embedded quantum annealer beats classical SA/SQA at a fraction of the wall time (Sec. 8.13).

11. Resource-aware penalty calibration (Sec. 8.7) provably never loosens the sufficient coefficients of the utility-scale baseline, tightens B/D by 23.8/32.8% on average on the 139 of 1,080 held-out instances where tightening binds, and—as a negative result—produces no detectable OpenJij SA/SQA quality change under fixed budgets.

11.1 Findings on the Implemented Extensions

The four extensions implemented in Sec. 10 add the following conclusions to the core results:

1. **Decoherence-aware streaming** (Sec. 10.1) lifts delivered fidelity by using a fidelity-risk term whose hold model matches the evaluation metric, at a controllable utility cost.

2. **Hard-constraint MPS** (Sec. 10.2) removes the feasibility-repair pass and penalty tuning while matching or beating penalty-MPS quality and runtime.

3. **Q-learning** (Sec. 10.3) matches the annealer's routing quality with 25× cheaper online inference, transferring cost to offline training.

4. **Hardware profiles** (Sec. 10.4) show a 2.8× delivered-fidelity spread across current platform families, motivating coherence-aware scheduling on short-T_2 devices.

5. **Temporal-memory joint scheduling** (Sec. 4.6) shows that memory-agnostic admission overcommits: the joint router delivers every admitted request on time while the baseline admits pairs that have decohered—serving correctly dominates serving optimistically.

6. **Topology robustness** (Sec. 8.15) ranks the five generative families by served ratio, with small-world topologies easiest and hub-based Barabási–Albert hardest.

11.2 Remaining Future Directions

1. **Adaptive bond dimension:** let the MPS compressor grow χ on the fly when the truncation error stays above a tolerance, as suggested by the bond-dimension analysis (Sec. 8.2).

2. **Deep-RL generalization:** extend the tabular Q-learning router to function approximation so the policy generalizes across unseen topologies.

3. **Physical testbed deployment:** port the hardware profiles to measured platform parameters (real T_1/T_2, gate fidelities, and interface latencies) and validate end to end.
4. **Stochastic temporal windows:** extend the joint scheduler to probabilistic transmission success so that memory windows carry failure probabilities rather than determinism.
5. **Learned MPS ordering:** replace the heuristic request ordering in the MPS coupling graph with a learned permutation to improve bond efficiency on high-treewidth instances.
6. **Coefficient calibration on device:** the held-out calibration study (Sec. 8.7) is combinatorial and solver-free; porting the resource-aware bound to a live annealer would test whether the coefficient-tightening translates into wall-time or read-count savings at scale, and whether the "no solver-quality effect" negative result survives on larger instances and denser topologies.

References

[1] H. J. Kimble, "The quantum internet," *Nature*, vol. 453, pp. 1023–1030, 2008.

[2] S. Wehner, D. Elkouss, and R. Hanson, "Quantum internet: A vision for the road ahead," *Science*, vol. 362, no. 6412, p. eaam9288, 2018.

[3] C. Simon, "Towards a global quantum network," *Nat. Photonics*, vol. 11, pp. 678–680, 2017.

[4] H.-J. Briegel, W. Dür, J. I. Cirac, and P. Zoller, "Quantum repeaters: The role of imperfect local operations in quantum communication," *Phys. Rev. Lett.*, vol. 81, p. 5932, 1998.

[5] W. Dür, H.-J. Briegel, J. I. Cirac, and P. Zoller, "Quantum repeaters based on entanglement purification," *Phys. Rev. A*, vol. 59, p. 169, 1999.

[6] N. Sangouard, C. Simon, H. de Riedmatten, and N. Gisin, "Quantum repeaters based on atomic ensembles and linear optics," *Rev. Mod. Phys.*, vol. 83, p. 33, 2011.

[7] K. Azuma, S. E. Economou, D. Elkouss, P. Hilaire, L. Jiang, H.-K. Lo, and I. Tzitrin, "Quantum repeaters: From quantum networks to the quantum internet," *Rev. Mod. Phys.*, vol. 95, p. 045006, 2023.

[8] R. Horodecki, P. Horodecki, M. Horodecki, and K. Horodecki, "Quantum entanglement," *Rev. Mod. Phys.*, vol. 81, pp. 865–942, 2009.

[9] R. F. Werner, "Quantum states with Einstein–Podolsky–Rosen correlations admitting a hidden-variable model," *Phys. Rev. A*, vol. 40, pp. 4277–4281, 1989.

[10] R. Jozsa, "Fidelity for mixed quantum states," *J. Mod. Opt.*, vol. 41, no. 12, pp. 2315–2323, 1994.

[11] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein–Podolsky–Rosen channels," *Phys. Rev. Lett.*, vol. 70, p. 1895, 1993.

[12] C. H. Bennett, G. Brassard, S. Popescu, B. Schumacher, J. A. Smolin, and W. K. Wootters, "Purification of noisy entanglement and faithful teleportation via noisy channels," *Phys. Rev. Lett.*, vol. 76, p. 722, 1996.

[13] A. K. Ekert, "Quantum cryptography based on Bell's theorem," *Phys. Rev. Lett.*, vol. 67, p. 661, 1991.

[14] N. Gisin, G. Ribordy, W. Tittel, and H. Zbinden, "Quantum cryptography," *Rev. Mod. Phys.*, vol. 74, pp. 145–195, 2002.

[15] V. Scarani, H. Bechmann-Pasquinucci, N. J. Cerf, M. Dušek, N. Lütkenhaus, and M. Peev, "The security of practical quantum key distribution," *Rev. Mod. Phys.*, vol. 81, pp. 1301–1350, 2009.

[16] S. Pirandola, U. L. Andersen, L. Banchi, M. Berta, D. Bunandar, R. Colbeck, D. Englund, T. Gehring, C. Lupo, C. Ottaviani, J. L. Pereira, M. Razavi, J. S. Shaari, M. Tomamichel, V. C. Usenko, G. Vallone, P. Villoresi, and P. Wallden, "Advances in quantum cryptography," *Adv. Opt. Photon.*, vol. 12, p. 1012, 2020.

[17] R. Van Meter, *Quantum Networking*. Wiley, 2014.

[18] M. Pant, H. Krovi, D. Towsley, L. Tassiulas, L. Jiang, P. Basu, D. Englund, and S. Guha, "Routing entanglement in the quantum internet," *npj Quantum Information*, vol. 5, p. 25, 2019.

[19] M. Caleffi, "Optimal routing for quantum networks," *IEEE Access*, vol. 5, pp. 22299–22312, 2017.

[20] L. Gyongyosi and S. Imre, "Entanglement-gradient routing of quantum networks," *Sci. Rep.*, vol. 9, p. 19274, 2019.

[21] M. Cuquet and J. Calsamiglia, "Entanglement percolation in quantum complex networks," *Phys. Rev. Lett.*, vol. 103, p. 240503, 2009.

[22] E. Schoute, L. Mancinska, T. Islam, A. Kahanamoku-Meyer, and S. Wehner, "Shortcuts to quantum network routing," arXiv:1610.05238, 2016.

[23] W. Kozlowski, A. Dahlberg, and S. Wehner, "Designing a quantum network protocol," in *Proc. ACM CoNEXT*, 2020.

[24] T. Coopmans, R. Knegjens, A. Dahlberg, D. Maier, L. Nijsten, A. N. de Oliveira, M. Papendrecht, J. Rabbie, F. Rozpędek, M. Skrzypczyk, L. Wubben, W. de Jong, D. Podareanu, A. Torres-Knoop, D. Elkouss, and S. Wehner, "NetSquid, a NETwork SIMulator for Quantum Information using Discrete events," *Commun. Phys.*, vol. 4, p. 164, 2021.

[25] X. Wu, A. Kolar, J. Chung, D. Jin, T. Zhong, R. Kettimuthu, and M. Suchara, "SeQUeNCe: A customizable discrete-event simulator of quantum networks," *Quantum Sci. Technol.*, vol. 6, p. 045027, 2021.

[26] T. Kadowaki and H. Nishimori, "Quantum annealing in the transverse Ising model," *Phys. Rev. E*, vol. 58, p. 5355, 1998.

[27] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, p. 671, 1983.

[28] M. W. Johnson, M. H. S. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, E. M. Chapple, C. Enderud, J. P. Hilton, K. Karimi, E. Ladizinsky, N. Ladizinsky, T. Oh, I. Perminov, C. Rich, M. C. Thom, E. Tolkacheva, C. J. S. Truncik, S. Uchaikin, J. Wang, B. Wilson, and G. Rose, "Quantum annealing with manufactured spins," *Nature*, vol. 473, pp. 194–198, 2011.

[29] S. Boixo, T. F. Rønnow, S. V. Isakov, Z. Wang, D. Wecker, D. A. Lidar, J. M. Martinis, and M. Troyer, "Evidence for quantum annealing with more than one hundred qubits," *Nat. Phys.*, vol. 10, pp. 218–224, 2014.

[30] S. E. Venegas-Andraca, W. Cruz-Santos, C. McGeoch, and M. Lanzagorta, "A cross-disciplinary glimpse at practical quantum annealing," *Int. J. Circuit Theory Appl.*, vol. 47, pp. 853–883, 2019.

[31] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, p. 5, 2014.

[32] F. Glover, G. Kochenberger, and Y. Du, "A tutorial on formulating and using QUBO models," *4OR*, vol. 17, pp. 335–369, 2019.

[33] K. Tanahashi, S. Takayanagi, T. Motohashi, and S. Tanaka, "Application of Ising machines and a software development methodology to portfolio optimization," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 5, pp. 2604–2616, 2022.

[34] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv:1411.4028, 2014.

[35] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[36] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *J. Chem. Phys.*, vol. 21, p. 1087, 1953.

[37] OpenJij: An open-source framework for quantum and simulated annealing. `https://openjij.github.io/`

[38] S. R. White, "Density matrix formulation for quantum renormalization groups," *Phys. Rev. Lett.*, vol. 69, p. 2863, 1992.

[39] G. Vidal, "Efficient classical simulation of slightly entangled quantum computations," *Phys. Rev. Lett.*, vol. 91, p. 147902, 2003.

[40] U. Schollwöck, "The density-matrix renormalization group in the age of matrix product states," *Ann. Phys.*, vol. 326, pp. 96–192, 2011.

[41] R. Orús, "A practical introduction to tensor networks: Matrix product states and projected entangled pair states," *Ann. Phys.*, vol. 349, pp. 117–158, 2014.

[42] R. Orús, "Tensor networks for complex quantum systems," *Nat. Rev. Phys.*, vol. 1, pp. 538–550, 2019.

[43] T. Hao, X. Huang, R. Jia, and C. Peng, "A quantum-inspired tensor network algorithm for constrained combinatorial optimization problems," *Frontiers in Physics*, vol. 10, 2022, arXiv:2203.15246.

[44] K. Nakada, K. Tanahashi, and S. Tanaka, "Quick design of feasible tensor networks for constrained combinatorial optimization," *Quantum*, vol. 9, p. 1799, 2025.

[45] F. Preisser, O. Mc Keever, and M. Lubasch, "Variational matrix product states for combinatorial optimization," *Phys. Rev. Research*, vol. 8, p. 033027, 2026.

[46] A. Mata Ali, "Explicit solution equation for every combinatorial problem via tensor networks: MeLoCoToN," arXiv:2502.05981, 2025.

[47] A. Mata Ali, L. Delgado, A. Roura, and I. de Leceta, "Polynomial-time solver of tridiagonal QUBO, QUDO and tensor QUDO problems with tensor networks," arXiv:2309.10509, 2023.

[48] F. Rosales, "Quantum-inspired optimization for adaptive multi-demand routing in QKD networks," arXiv:2605.27425, 2025.

[49] "Q-Learning for Resource-Aware and Adaptive Routing in Trusted-Relay QKD Networks," 2025.

[50] J. A. Boyan and M. Littman, "Packet routing in dynamically changing networks: A reinforcement learning approach," in *Proc. NeurIPS*, 1993.

[51] B. Mao, F. Fadlullah, N. Kato, F. Tang, and T. Onouchi, "Routing or computing? The paradigm shift towards intelligent computer network packet transmission based on deep learning," *IEEE Trans. Computers*, vol. 66, pp. 1946–1960, 2017.

[52] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet: Learning optimized routes with traffic flows as messages," *IEEE J. Sel. Areas Commun.*, vol. 38, pp. 192–205, 2020.

[53] E. Koutsoupias and C. Papadimitriou, "Worst-case equilibria," in *Proc. STACS*, 1999, pp. 404–413.

[54] T. Roughgarden and E. Tardos, "How bad is selfish routing?" *J. ACM*, vol. 49, pp. 236–259, 2002.

[55] D. Braess, "Über ein Paradoxon aus der Verkehrsplanung," *Unternehmensforschung*, vol. 12, pp. 258–268, 1968.

[56] D. Bertsimas and M. Sim, "The price of robustness," *Operations Research*, vol. 52, pp. 35–53, 2004.

[57] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley, 2001.

[58] A. Charnes and W. W. Cooper, "Chance-constrained programming," *Management Science*, vol. 6, no. 1, pp. 73–79, 1959.

[59] A. Prékopa, *Stochastic Programming*. Kluwer, 1995.

[60] J. R. Birge and F. Louveaux, *Introduction to Stochastic Programming*. Springer, 1997.

[61] A. Ben-Tal, L. El Ghaoui, and A. Nemirovski, *Robust Optimization*. Princeton University Press, 2009.

[62] A. I. Lvovsky, B. C. Sanders, and W. Tittel, "Optical quantum memory," *Nat. Photonics*, vol. 3, pp. 706–714, 2009.